A

REPORT

ON

DocuMind AI: Secure Per-User RAG-Based Intelligent Document Q&A System

BY

M.Srichakra Kanwa

(Roll No: 23STUCHH010756)

Project Report submitted to IcfaiTech as a partial fulfillment of the requirements for the award of the Degree of B.Tech in Computer Science & Engineering

Under the supervision of

DR. K. Bhargavi

Assistant Professor, Dept. of CSE

Department of Computer Science & Engineering

Faculty of Science & Technology, IFHE University

Hyderabad - 501203

April 2026

DECLARATION

I, M.Srichakra Kanwa (Roll No. 23STUCHH010756), student of B.Tech in Computer Science & Engineering, Faculty of Science & Technology, IFHE University, Hyderabad, hereby declare that the project report entitled "DocuMind AI: Secure Per-User RAG-Based Intelligent Document Q&A System" submitted in partial fulfillment of the requirements for the award of the degree of B.Tech in Computer Science & Engineering is a record of original work done by me during the academic year 2025-2026, under the supervision of DR. K. Bhargavi, Assistant Professor, Department of Computer Science & Engineering, IFHE University.

I further declare that this project report has not been submitted for the award of any degree or diploma in any other university or institution. The project work is based on my own investigation and no part of it has been published or submitted for publication elsewhere.

Date: April 2026

M.Srichakra Kanwa
(Signature of Student)

DR. K. Bhargavi
(Signature of Guide)

CERTIFICATE

This is to certify that the project report entitled "DocuMind AI: Secure Per-User RAG-Based Intelligent Document Q&A System" submitted by M.Srichakra Kanwa (Roll No. 23STUCHH010756), student of B.Tech, Computer Science & Engineering, Faculty of Science & Technology, IFHE University, Hyderabad, is a record of bonafide project work carried out under my supervision during the academic year 2025-2026.

The report has been reviewed and found satisfactory in scope and quality for submission as partial fulfillment of the requirements for the award of the degree of B.Tech in Computer Science & Engineering. This work has not been submitted to any other university or institution for any degree or diploma.

Project Supervisor:

DR. K. Bhargavi
Assistant Professor
Dept. of CSE, IFHE University

Head of Department:

DR. P. Rohini
Professor & HOD
Dept. of CSE, IFHE University

ABSTRACT

The rapid growth in digital documents, particularly PDFs, has created an urgent need for intelligent systems capable of understanding and responding to user queries from large document collections. Traditional keyword-based search tools lack semantic understanding, while general-purpose AI chatbots frequently produce hallucinated responses not grounded in actual document content.

This project presents DocuMind AI, a secure, per-user Retrieval-Augmented Generation (RAG) based intelligent document question-answering system. The system enables authenticated users to upload PDF documents and pose natural language questions, receiving accurate, context-grounded answers with Markdown formatting and source citations. The retrieval-constrained design significantly reduces hallucination by ensuring that all responses are derived from the uploaded document content.

The system integrates PyMuPDF and Tesseract OCR for hybrid text extraction, SentenceTransformers (all-MiniLM-L6-v2) for 384-dimensional dense vector embeddings, FAISS IndexFlatIP for per-user vector indexing and semantic retrieval, and Google Gemini LLM for grounded answer generation with multi-key rotation. The backend is built on FastAPI with JWT authentication and SQLAlchemy ORM. The frontend is a React 18 single-page application featuring chat sessions, a split-pane document viewer, source citations, dark/light theme toggle, and OTP-based password reset via Gmail SMTP.

The system was validated through 25 functional test cases covering authentication, document processing, RAG retrieval, cross-user security isolation, and browser compatibility. All test cases passed successfully. The report is structured across nine chapters covering the problem statement, literature review, system architecture, requirements, implementation, API reference, testing, limitations, and conclusions.

Keywords: *Retrieval-Augmented Generation, Document Q&A, FAISS, Vector Embeddings, OCR, LLM, FastAPI, React, Google Gemini, JWT Authentication, SentenceTransformers, Per-User Isolation, Hallucination Mitigation*

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project mentor DR. K. Bhargavi for her invaluable guidance, continuous support, and constructive feedback throughout the development of this project. Her expert insights into NLP, retrieval systems, and full-stack web development significantly shaped the direction and quality of this work. Her encouragement during challenging phases of development was instrumental in bringing this project to successful completion.

I am deeply thankful to DR. P. Rohini, Head of the Department, Faculty of Science & Technology, IFHE University, for providing the academic framework, encouragement, and resources necessary for undertaking this project. Her leadership and commitment to research excellence have been a constant source of motivation.

I extend my heartfelt thanks to all the faculty members of the Department of Computer Science at IcfaiTech, Hyderabad, whose teachings in data structures, algorithms, machine learning, database management, and web technologies provided the foundational knowledge that directly contributed to the successful completion of this project. The rigorous academic curriculum at IFHE University provided the technical depth required to design and implement a production-quality AI system.

I am also grateful to the developers and maintainers of the open-source libraries and frameworks that powered this project, including FastAPI, React, FAISS, PyMuPDF, LangChain, SentenceTransformers, and Tesseract OCR. Their contributions to the open-source community made this advanced AI-powered document platform possible.

Finally, I would like to thank my family and friends for their unwavering support, patience, and encouragement during the long hours of development and writing. Their belief in my abilities gave me the confidence to tackle complex technical challenges and produce a report that reflects the full scope of this project.

M.Srichakra Kanwa

23STUCHH010756, B.Tech CSE

IFHE University, Hyderabad

TABLE OF CONTENTS

Declaration.....i
Certificate.....ii
Abstract.....iii
Acknowledgement.....iv
Table of Contents.....v
List of Figures.....vii
List of Tables.....viii
1. Introduction.....1
1.1 Background and Motivation..... 1
1.2 Problem Statement..... 1
1.3 Proposed Solution..... 2
1.4 Aim of the Project..... 3
1.5 Goals of the Project..... 3
1.6 Scope of the Project..... 4
1.7 Process Flow..... 4
1.8 Report Organization.....5
2. Literature Review.....7
2.1 Large Language Models and Transformer Architecture.....7
2.2 Retrieval-Augmented Generation..... 8
2.3 Dense Vector Retrieval and Semantic Search.....9
2.4 Optical Character Recognition for Scanned Documents..... 10
2.5 Text Chunking and Segmentation Strategies.....10
2.6 Web Application Frameworks.....11
2.7 Security in Web Applications.....12
2.8 Comparative Analysis of Existing Systems..... 12
2.9 Research Gaps and Motivation..... 13
2.10 Transformer Architecture: Mathematical Foundations.....13
2.11 Vector Database Technologies..... 14
2.12 NLP for Document Question Answering..... 15
3. System Architecture.....16
3.1 Overall Architecture.....16

3.2 Authentication and Authorization Flow....16
3.3 RAG Pipeline Architecture.... 17
3.4 Per-User Data Isolation Architecture....18
3.5 Document Upload and Indexing Sequence.... 19
3.6 Security Architecture.... 19
3.7 Frontend Application Architecture.... 20
3.8 Deployment Architecture.... 21
4. System Requirements....22
4.1 Hardware Requirements....22
4.2 Software Requirements.... 22
4.3 Python Package Dependencies.... 22
4.4 Environment Configuration.... 23
5. Implementation....25
5.1 Backend Server and Application Entry Point.... 25
5.2 Authentication Module....26
5.3 OTP-Based Password Reset Module.... 30
5.4 Document Processing and Hybrid OCR Pipeline.... 33
5.5 Vector Indexing with FAISS.... 34
5.6 Question-Answering Chain and LLM Integration.... 37
5.7 Database Models.... 40
5.8 Frontend Application.... 42
5.9 Chat Response and Source Citations....49
5.10 Document Management Page.... 50
5.11 Document Viewer - Split-Pane Interface....52
5.12 User Profile and Account Settings.... 53
5.13 Deployment Configuration....55
5.14 Document Deletion and Index Cleanup.... 56
5.15 Branded HTML Email for OTP Delivery.... 57
6. API Reference....59
6.1 Authentication Endpoints....59
6.2 Document Endpoints.... 59
6.3 Question-Answering and Session Endpoints.... 59
6.4 Request and Response Schemas....60
7. Testing and Results.... 61
7.1 Testing Methodology....61

7.2 Performance Benchmarks..... 61
7.3 Functional Test Cases.....62
7.4 Security Test Results.....64
7.5 Browser Compatibility Testing.....64
7.6 Key Design Decisions..... 65
7.7 Project Statistics..... 66
8. Limitations of the System.....67
8.1 External API Dependency.....67
8.2 Scalability Constraints..... 67
8.3 OCR Quality and Language Limitations.....68
8.4 Retrieval and Generation Quality Limitations..... 68
8.5 Collaboration and Multi-User Limitations.....68
9. Conclusion and Future Scope.....70
9.1 Conclusion.....70
9.2 Future Scope.....71
References.....74

LIST OF FIGURES

Figure 3.1: High-Level Three-Tier System Architecture 16
Figure 3.2: Authentication and API Authorization Sequence 17
Figure 3.3: Complete RAG Pipeline - Indexing and Retrieval Phases 18
Figure 3.4: Per-User Storage Directory Structure and Isolation Levels 19
Figure 3.5: Document Upload and Async Indexing Sequence Diagram 19
Figure 5.1: Login Page - Username/Email Input with Theme Toggle Button 29
Figure 5.2: Registration Page - Three-Field Form with Validation 30
Figure 5.3: Password Reset Step 1 - Enter Registered Email Address 32
Figure 5.4: Password Reset Step 2 - Enter 6-Digit OTP Received by Email 33
Figure 5.5: Hybrid PDF Text Extraction Decision Flow 34
Figure 5.6: FAISS Indexing Pipeline and Diversity-Aware Retrieval 37
Figure 5.7: Question-Answering Chain Processing Flow 38
Figure 5.8: Gemini API Key Rotation with Per-Key Cooldown Timers 39
Figure 5.9: Chat Page - Dark Theme with Session Sidebar and Query Chips 44
Figure 5.10: Chat Page - Light Theme Interface 45
Figure 5.11: PDF Tools Dropdown - 15 External Tools Accessible via Hover 46
Figure 5.12: User Profile Dropdown - Navigation to Profile and Documents 46
Figure 5.13: Chat Response with Source Citation Pill-Buttons 50
Figure 5.14: File Upload Dialog with Drag-and-Drop Zone 51
Figure 5.15: Documents Page with Status Badges and File Table 52
Figure 5.16: Document Viewer with Left PDF Pane and Right Q&A Sidebar 53
Figure 5.17: Profile Page - Profile Tab with Account Information 54
Figure 5.18: Profile Page - Change Password Tab 54
Figure 5.19: Profile Page - Account Tab with Usage Statistics 55

LIST OF TABLES

Table 2.1: Comparison of RAG Retrieval Strategies 9
Table 2.2: Comparison of DocuMind AI with Existing Systems 12
Table 2.3: Comparison of Vector Store Options 14
Table 4.1: Hardware Requirements 22
Table 4.2: Software Requirements 22
Table 6.1: Authentication Endpoints 59
Table 6.2: Document Management Endpoints 59
Table 6.3: Q&A and Session Endpoints 59
Table 7.1: Performance Benchmark Results 61
Table 7.2: Functional Test Cases (Authentication Module) 62
Table 7.3: Functional Test Cases (Document Processing Module) 62
Table 7.4: Functional Test Cases (Q&A and Session Module) 63
Table 7.5: Functional Test Cases (OTP and Cross-Feature Module) 63
Table 7.6: Cross-User Security Test Matrix 64
Table 7.7: Browser Compatibility Test Results 64
Table 7.8: DocuMind AI Project Statistics 66
Table 8.1: Summary of System Limitations and Mitigations 69

1. Introduction

1.1 Background and Motivation

Digital documents have become the primary medium for preserving and sharing knowledge across academic, legal, corporate, and governmental domains. Whether it is a research paper, a legal contract, a corporate manual, or a university textbook, a significant proportion of critical information resides in PDF files, often spanning hundreds of densely formatted pages.

The global volume of digital documents continues to grow at an unprecedented rate. According to industry estimates, over 2.5 trillion PDF documents exist digitally, with millions more created daily across enterprise, academic, and government sectors. Organizations and individuals alike face the challenge of efficiently navigating this information, extracting meaningful insights, and making decisions based on document content.

While this shift to digital documentation has facilitated easier storage and distribution, it has introduced a fundamental challenge: extracting specific answers from large document collections remains a time-consuming and cognitively demanding process. A student preparing from a 200-page textbook, a lawyer reviewing a 300-page contract, or a researcher cross-referencing multiple journal papers all encounter the same limitation with conventional tools: the inability to pose questions in natural language and receive accurate, contextually grounded answers.

Conventional PDF applications rely entirely on keyword matching, which means that a search for "revenue decline" would not surface a passage stating "falling profits." This semantic gap between user intent and literal text matching represents a significant productivity barrier. The emergence of Large Language Models (LLMs) such as Google Gemini and GPT-4 offered a potential solution: these models can understand and generate natural language with high fidelity. However, general-purpose LLMs trained on fixed datasets have no knowledge of user-specific documents uploaded after training. When queried about unseen content, they frequently hallucinate, producing plausible but factually incorrect responses. Retrieval-Augmented Generation (RAG) resolves this by retrieving relevant document sections and grounding LLM responses in actual document content, significantly reducing hallucination.

The motivation for DocuMind AI arises from this convergence of need and opportunity: a production-quality, privacy-first, per-user RAG system that is accessible to end-users without requiring technical expertise, and that operates on the user's own documents with strict data isolation.

1.2 Problem Statement

Despite the availability of powerful LLMs, document processing tools, and open-source vector databases, no accessible, privacy-preserving, per-user document Q&A system

simultaneously satisfies all of the following requirements:

- Support for both digital and scanned PDFs through hybrid text extraction with automatic OCR fallback for image-based pages.
- Strict per-user data isolation ensuring that one user's uploaded documents, vector indices, and conversation history are never accessible to another user.
- Minimized hallucination through retrieval-constrained generation, where all LLM responses are grounded in retrieved document content.
- Efficient semantic search using dense vector embeddings across multiple documents owned by a single user.
- A modern, responsive user interface with dark/light themes, persistent chat session management, and Markdown rendering of AI responses.
- Branded OTP-based password reset via email for secure account recovery without administrator intervention.
- File storage on a secure backend server (not the user's local device), with complete privacy guarantees.
- Production-ready deployment with configurable environment settings for both development and cloud environments.

Existing solutions such as ChatPDF address some of these requirements but fail on privacy (shared infrastructure, no per-user isolation) and cannot process scanned documents. Enterprise solutions such as Adobe AI are proprietary and cost-prohibitive for individual use. Open-source frameworks such as LlamaIndex provide the building blocks but are developer tools lacking user-facing applications with complete authentication, document management, and deployment configurations. DocuMind AI was designed to bridge this gap.

1.3 Proposed Solution

DocuMind AI is a full-stack self-hostable RAG system designed for individual users who require private, accurate, and efficient document question-answering. The system is built as a two-tier web application: a Python FastAPI backend serving a React 18 frontend, with all AI processing, document storage, and vector indexing performed on the backend server.

The core design philosophy is document-grounded generation: the LLM is restricted to generating answers based exclusively on retrieved document passages. This is enforced through strict system prompts that explicitly instruct the model to acknowledge when information is not available in the provided context. Combined with per-user FAISS indices that search only the authenticated user's documents, this approach ensures that answers are always grounded in the user's actual content.

The architecture scales from a single-user local deployment (SQLite) to a multi-user cloud deployment (PostgreSQL on Railway) without code changes, relying only on environment variable configuration. The key technical components are integrated as follows:

- FastAPI backend with JWT authentication, bcrypt password hashing, and SQLAlchemy ORM providing secure, scalable API services.
- PyMuPDF combined with Tesseract OCR providing hybrid PDF text extraction for both digital and scanned documents.
- SentenceTransformers (all-MiniLM-L6-v2) generating 384-dimensional dense vector embeddings for semantic search.
- FAISS IndexFlatIP providing per-user exact cosine similarity search with diversity-aware chunk deduplication.
- Google Gemini LLM with multi-key rotation, automatic continuation, and strict document-grounding system prompts.
- React 18 with Vite 5 providing a split-pane document viewer, source citations, Markdown rendering, and theme management.
- Branded HTML OTP email via Gmail SMTP providing secure, self-service password reset.
- Railway (backend) and Vercel (frontend) deployment configurations for zero-configuration cloud deployment.

1.4 Aim of the Project

The primary aim of this project is to design, develop, and deploy a secure, per-user Retrieval-Augmented Generation system that enables authenticated users to upload PDF documents and interactively ask natural language questions. The system aims to provide accurate, document-grounded answers with significantly reduced hallucination, presented in rich Markdown format with verifiable source citations.

A secondary aim is to demonstrate that a production-quality, full-stack RAG application can be built and deployed using open-source tools and free-tier cloud services, making advanced document intelligence accessible to individual users without expensive proprietary solutions or dedicated GPU infrastructure. The project also aims to establish best practices for per-user data isolation and multi-key LLM API management in production RAG systems.

1.5 Goals of the Project

1. Build a full-stack document Q&A system with secure per-user authentication (JWT) and strict data isolation at the filesystem, database, and vector index levels.
2. Support both digital and scanned PDFs using hybrid text extraction with OCR fallback, without requiring external PDF rendering dependencies.
3. Implement efficient semantic search using dense vector embeddings (SentenceTransformers) and FAISS indexing with diversity-aware chunk deduplication.
4. Integrate Google Gemini LLM with multi-key rotation, automatic response

continuation, and strict document-grounded system prompts.

5. Provide chat session management with follow-up question detection, conversation context memory, and session rename/delete functionality.
6. Design a modern React frontend with dark/light themes, custom Markdown rendering, split-pane document viewer, and PDF tools dropdown.
7. Implement branded HTML OTP-based password reset via Gmail SMTP with 10-minute expiry and single-use enforcement.
8. Configure Railway (backend) and Vercel (frontend) deployment with environment-based configuration for local and cloud environments.
9. Achieve a 100% pass rate on 25 functional test cases covering authentication, document processing, RAG retrieval, security isolation, and browser compatibility.

1.6 Scope of the Project

DocuMind AI is scoped as a personal document intelligence platform for individual users operating in private workspaces. Within scope is multi-document upload and management per user account, English-language PDF querying, single-user private workspaces, and deployment to both local development and cloud environments.

Out of scope for the current version are: real-time multi-user collaborative editing, non-English language document querying, non-PDF document formats (DOCX, PPTX, Excel), on-device LLM inference, and GPU-accelerated vector search. These are identified as future enhancements in Chapter 9.

1.7 Process Flow

The end-to-end process flow of DocuMind AI, from user registration through receiving a document-grounded answer, proceeds through the following eleven steps:

1. User Registration: User submits username, email, and password. The backend validates uniqueness, hashes the password with bcrypt, and stores the user record in the database.
2. JWT Login: User submits credentials. The backend verifies the bcrypt hash and issues a JWT bearer token (HS256-signed, 7-day expiry) returned to the client.
3. PDF Upload: User selects a PDF file. The backend saves it to storage/{user_id}/files/ with a UUID prefix and returns a document ID with "Processing" status.
4. Background Indexing: A FastAPI BackgroundTask runs asynchronously: PyMuPDF extracts digital text; Tesseract OCR processes scanned pages. The combined text is chunked and embedded.
5. FAISS Indexing: SentenceTransformer encodes each chunk into a 384-dim vector. Vectors are added to the user's FAISS IndexFlatIP. Chunk metadata is saved to chunks.json.

6. Status Update: The document record is updated from is_indexed=0 (Processing) to is_indexed=1 (Ready). The frontend polls every 2 seconds and updates the status badge.
7. Question Submission: User types a natural language question. The system classifies it (general/summary/compare/predict) to determine the retrieval depth K (5-15 chunks).
8. FAISS Retrieval: The question is embedded and searched against the user's FAISS index. The diversity-aware algorithm selects K unique, semantically relevant chunks.
9. Prompt Assembly: Retrieved chunks are formatted as document context. The strict RAG system prompt is prepended. Conversation history (last 3 turns) is included for follow-ups.
10. LLM Generation: The assembled prompt is sent to Google Gemini via the GeminiKeyRotator. Automatic continuation handles truncated responses (up to 3 rounds).
11. Answer Delivery: The Markdown-formatted answer and source filenames are returned to the React frontend, rendered with the custom MarkdownMessage component and source citation pills.

1.8 Report Organization

The remainder of this report is organized as follows:

- Chapter 2 (Literature Review): Reviews the key published research and technologies forming the theoretical foundation of DocuMind AI, including LLMs, RAG, dense retrieval, OCR, and web security.
- Chapter 3 (System Architecture): Presents the overall three-tier architecture, RAG pipeline design, per-user data isolation model, security architecture, and deployment topology.
- Chapter 4 (System Requirements): Specifies the hardware, software, Python package, and environment configuration requirements for running DocuMind AI.
- Chapter 5 (Implementation): Provides detailed source code excerpts, design explanations, and screenshots for all major components including authentication, document processing, vector indexing, LLM integration, and the React frontend.
- Chapter 6 (API Reference): Documents all 22 RESTful API endpoints with methods, routes, descriptions, and sample request/response bodies.
- Chapter 7 (Testing and Results): Presents a comprehensive testing methodology, 40 test cases, performance benchmarks, security testing results, and key design decisions.
- Chapter 8 (Limitations): Documents the known limitations of the system including API dependency, scalability constraints, OCR quality, and language support.
- Chapter 9 (Conclusion and Future Scope): Summarizes the achievements of the project and presents a structured roadmap for future enhancements.

- References: Lists all cited academic papers, technical documentation, and online resources in IEEE format.

2. Literature Review

This chapter reviews the key technologies, published research, and existing systems that form the theoretical and technical foundation of DocuMind AI. The review is organized by topic area, beginning with the mathematical foundations of language models and progressing through retrieval systems, document processing, web frameworks, and security. Each section concludes with relevance to DocuMind AI's implementation.

2.1 Large Language Models and Transformer Architecture

The Transformer architecture introduced by Vaswani et al. [1] in 2017 represented a fundamental paradigm shift in sequence modeling. Prior to Transformers, state-of-the-art sequence models relied on recurrent neural networks (LSTMs and GRUs) that processed tokens sequentially. This sequential processing prevented parallelization during training and created difficulties in capturing long-range dependencies due to gradient degradation.

The Transformer replaced recurrence with multi-head self-attention, enabling all positions in the sequence to attend to each other simultaneously. The scaled dot-product attention mechanism is computed as follows:

```
Attention(Q, K, V) = softmax(Q * K^T / sqrt(d_k)) * V

Where:
  Q (Query)  : Representation of the current token
  K (Key)    : Representations of all tokens being attended to
  V (Value)  : Content vectors to be aggregated
  d_k : Dimensionality of the key vectors (scaling factor)

Multi-Head Attention:
  MultiHead(Q, K, V) = Concat(head_1, ..., head_h) * W_O
  head_i = Attention(Q * W_Q_i, K * W_K_i, V * W_V_i)

  h = 8 heads in original Transformer
  Each head projects to d_k = d_model / h dimensions
```

Devlin et al. [2] subsequently developed BERT (Bidirectional Encoder Representations from Transformers), demonstrating that a model pre-trained on large corpora using Masked Language Modeling (MLM) and Next Sentence Prediction (NSP) tasks achieves state-of-the-art results across 11 NLP benchmarks when fine-tuned. The key innovation was that pre-training on general text followed by task-specific fine-tuning vastly outperformed training from scratch on task-specific data alone.

Brown et al. [3] demonstrated with GPT-3 (175B parameters) that sufficiently large autoregressive language models can perform few-shot tasks directly from natural language prompt examples without any gradient updates. This enabled API-based LLM access, where developers can instruct models through prompting alone. Google Gemini [4] extends this paradigm with native multi-modal capabilities and a one-million-token context window.

DocuMind AI uses the gemini-2.5-flash variant, chosen for its balance of response quality, API cost, and throughput at the free tier.

The hallucination problem is one of the most critical limitations of modern LLMs for practical applications. Maynez et al. [5] demonstrated through human evaluation that abstractive summarization models regularly produce intrinsically hallucinated statements contradicting the source text. Ji et al. [6] conducted a comprehensive survey classifying hallucinations into two categories: intrinsic hallucination (contradicting the source material) and extrinsic hallucination (generating facts not present in any source). This limitation is particularly problematic for document Q&A applications, where users require verifiable, accurate answers from their documents rather than plausible-sounding fabrications.

2.2 Retrieval-Augmented Generation

The Retrieval-Augmented Generation (RAG) framework, formally introduced by Lewis et al. [8] at NeurIPS 2020, proposes a principled approach to addressing LLM hallucination and knowledge staleness. The core insight is that if the model does not possess the required knowledge in its parametric memory, relevant passages should be retrieved from an external knowledge source and provided as non-parametric context at inference time.

The formal RAG formulation defines a probability over generated sequences conditioned on both the input query and retrieved passages:

```
RAG-Sequence model:
  p(y|x) = sum_z [ p_eta(z|x) * prod_i p_theta(y_i | x, z, y_1:i-1) ]

Where:
  x   = input query
  z   = retrieved document passage
  y   = generated output sequence p_eta=
  retriever (e.g., DPR, FAISS) p_theta =
  generator (e.g., BART, Gemini)

DocuMind AI simplification (deterministic top-K):
  z* = top_K passages from FAISS search
  y   = Gemini(STRICT_PROMPT + z* + x)
```

Gao et al. [10] subsequently provided a taxonomy of RAG variants: Naive RAG (simple retrieve-then-generate), Advanced RAG (incorporating pre-retrieval query enhancement and post-retrieval re-ranking), and Modular RAG (flexible component substitution). DocuMind AI implements an Advanced RAG approach: pre-retrieval query classification adjusts the number of retrieved chunks (K) based on query type, and a diversity-aware post-retrieval deduplication step improves context coverage.

Asai et al. [11] proposed Self-RAG, which fine-tunes the LLM to generate special reflection tokens indicating whether retrieval is needed, whether retrieved passages are relevant, and whether the generated response is grounded. DocuMind AI achieves a comparable outcome through careful prompt engineering: the system prompt explicitly instructs the model to state

when the answer is not found in the provided context, and empirical testing confirmed this substantially reduces unfounded responses.

A practical consideration encountered during implementation was context window size management. When 15 chunks of 1800 characters each are assembled, the context string approaches 27,000 characters. The system caps context at 28,000 characters as a safety margin, though the Gemini 1.5 Flash model supports over one million tokens. This cap serves as a defensive measure for future model compatibility and API cost management.

Table 2.1: Comparison of RAG Retrieval Strategies

Strategy	Chunks (K)	Use Case	DocuMind AI Query Types
Precise Retrieval	3-5	Single-hop factual Q&A	"general", "explain"
Broad Retrieval	10-15	Multi-hop / summarization	"summary", "predict"
Comparative	8-12	Cross-document comparison	"compare", "vs"
Diversity-Weighted	4K candidates	Dedup to final K	All query types

2.3 Dense Vector Retrieval and Semantic Search

The shift from sparse to dense retrieval represents a fundamental transition in information retrieval. Traditional systems such as TF-IDF and BM25 represent documents and queries as high-dimensional sparse vectors (one dimension per vocabulary term), relying on lexical overlap. This causes the vocabulary mismatch problem: semantically equivalent terms that share no common tokens receive zero relevance scores.

Dense retrieval converts both queries and document passages into low-dimensional dense vectors (embeddings) where semantic similarity corresponds to geometric proximity. Karpukhin et al. [13] demonstrated with Dense Passage Retrieval (DPR) that this approach substantially outperforms BM25 for open-domain question answering. The similarity between a query embedding q and a passage embedding p is computed as:

```
Cosine Similarity:
  sim(q, p) = (q . p) / (|q| * |p|)

For L2-normalized vectors (|q|=|p|=1):
  sim(q, p) = q . p (inner product)

FAISS IndexFlatIP (Inner Product) = exact cosine similarity
over L2-normalized embeddings

Range: -1 (opposite) to +1 (identical)
Threshold for retrieval: sim > 0.3 implies semantic relevance
```

DocuMind AI employs SentenceTransformers [14], specifically the all-MiniLM-L6-v2 model. This model is a 6-layer, 22-million-parameter distilled MiniLM model producing 384-dimensional sentence embeddings. It was trained using contrastive learning on over one billion sentence pairs from diverse corpora including Wikipedia, news articles, and academic

papers. According to the MTEB benchmark [12], all-MiniLM-L6-v2 achieves an average score of 56.26 across semantic similarity, retrieval, and clustering tasks, with inference speeds of approximately 14,000 sentences per second on CPU.

For the vector index, FAISS (Facebook AI Similarity Search) [15] is used, specifically IndexFlatIP. FAISS implements exact inner product search over the full set of stored vectors. For the typical DocuMind AI use case of several thousand chunks per user, this exact search completes in sub-millisecond time. For enterprise-scale deployments, approximate nearest neighbor methods such as Inverted File (IVF) or Hierarchical Navigable Small World (HNSW) would be appropriate.

An implementation challenge encountered early in development was chunk redundancy in the retrieved results. Without quality controls, FAISS frequently returned multiple near-identical passages from the same paragraph of a document, resulting in a narrow and repetitive context window for the LLM. This was resolved by implementing a diversity-aware retrieval algorithm: the system retrieves 4x the target K candidates and filters out chunks whose normalized first-50-character signatures have been seen previously. This approach consistently improved the breadth and diversity of topics covered in the LLM context window.

2.4 Optical Character Recognition for Scanned Documents

A significant proportion of real-world PDF documents consist of scanned images rather than searchable digital text. Handwritten lecture notes, historical archives, and formally printed documents often exist purely as image-based PDFs. A document Q&A system that cannot process these documents has severely limited practical utility.

Smith [16] documented the evolution of Tesseract from a traditional pipeline OCR engine to a modern LSTM-based neural network system in Tesseract version 4+. The LSTM recognition engine (--oem 3) uses a recurrent neural network trained on a large multilingual dataset, significantly outperforming the legacy pattern matching approach on curved, degraded, and diverse fonts. The Page Segmentation Mode (--psm 6) treats the page as a single uniform block of text, appropriate for educational and technical documents.

DocuMind AI employs a hybrid extraction strategy. PyMuPDF (fitz) attempts digital text extraction on every page. A heuristic threshold of 80 characters is applied: pages yielding fewer than 80 extractable characters are classified as image-based and submitted for Tesseract OCR. This threshold was determined empirically: pages with only headers or footers (which are digital but text-sparse) average 60-70 characters, so 80 provides a safe boundary. The page is rendered at 200 DPI (a 2.78x zoom from 72 DPI) to provide sufficient resolution for the OCR engine, and the resulting PNG is processed in memory without writing to disk in the production implementation.

2.5 Text Chunking and Segmentation Strategies

A critical design decision in any RAG system is the strategy for splitting documents into chunks for indexing. This decision affects both retrieval precision and the quality of generated answers. Chunks that are too small lose contextual coherence: a half-sentence extracted from a paragraph provides insufficient context for meaningful generation. Chunks that are too large may exceed the embedding model's token limits (typically 256-512 subword tokens for miniature models) and dilute the semantic signal by containing multiple unrelated topics.

DocuMind AI uses LangChain's RecursiveCharacterTextSplitter [17], configured with a chunk_size of 1800 characters and chunk_overlap of 250 characters. The splitter attempts to split on separators in order of priority: paragraph breaks (double newline), single newlines, sentence-ending periods, word boundaries, and finally individual characters as a last resort. This hierarchy ensures semantically meaningful split points are preferred.

The 1800-character chunk size was selected after experimentation with sizes from 512 to 3000 characters. Chunks of 512 characters were too small for multi-sentence answers, frequently requiring many chunks to cover a single topic. Chunks of 3000 characters frequently crossed multiple topics, reducing the embedding's semantic focus. The 1800-character size, combined with 250-character overlap, consistently produced the highest answer quality during internal validation. The overlap ensures that sentences near a chunk boundary are fully captured in at least one adjacent chunk, preventing information loss at split points.

2.6 Web Application Frameworks

The choice of web framework significantly impacts development velocity, API quality, and production reliability. For the backend, FastAPI [18] was evaluated alongside Flask and Django REST framework. FastAPI was selected for three primary reasons: automatic request validation via Python type hints and Pydantic models (eliminating boilerplate validation code), native OpenAPI documentation generation at /docs and /redoc (critical for API iteration), and first-class asynchronous I/O support via Python's asyncio (essential for handling concurrent Gemini API calls without blocking).

FastAPI's dependency injection system (Depends()) was extensively used for authentication middleware (get_current_user), database session management (get_db), and rate limiting. This pattern cleanly separates cross-cutting concerns from business logic and enables straightforward unit test mocking of dependencies.

On the frontend, React [19] with Vite [20] was selected over Vue.js and Next.js. React's component model is well-suited for managing the complex state of the chat interface (sessions, messages, document selection, source citations). Vite's native ES module-based development server provides near-instant hot-module reload, significantly reducing frontend iteration time. React Router 6 [21] provides declarative nested routing with protected route components, enabling clean implementation of the authentication guard pattern.

2.7 Security in Web Applications

Security was treated as a first-class architectural concern rather than an afterthought. The authentication mechanism uses JSON Web Tokens (JWT) [22], specifically the HS256 (HMAC-SHA256) algorithm. JWTs provide a stateless, cryptographically verifiable authentication mechanism: the server does not need to maintain a session database, as the token itself contains a signed payload with the user ID and expiry time. The 7-day expiry balances security (limiting the window for compromised token exploitation) with usability (reducing the frequency of re-authentication).

Password storage uses bcrypt, a computationally intensive one-way hashing algorithm with a configurable cost factor. bcrypt's deliberate slowness (approximately 100ms per hash at the default cost factor) makes brute-force attacks computationally expensive. Ouyang et al. [7] demonstrated that instruction-following training improves safety in LLMs; DocuMind AI applies an analogous principle at the prompt level, instructing the model to decline generating responses that are not supported by retrieved context.

The OWASP Top 10 guidelines were reviewed during system design. SQL injection is prevented by SQLAlchemy's ORM parameterized queries. Broken access control is addressed by including user_id as a mandatory filter in all document and session queries. CORS headers are explicitly configured to prevent cross-origin attacks from unauthorized origins. File upload security is addressed by UUID filename prefixes, file type validation at the API level, and server-side storage (not client-accessible paths).

2.8 Comparative Analysis of Existing Systems

To contextualize the contributions of DocuMind AI, a survey of existing commercial and open-source document Q&A tools was conducted. Four representative systems were evaluated: ChatPDF (cloud-based document Q&A), Adobe AI (enterprise document intelligence), and LlamaIndex (open-source RAG developer framework). The comparison criteria were selected to reflect the system requirements enumerated in Section 1.2.

Table 2.2: Comparison of DocuMind AI with Existing Systems

Feature	DocuMind AI	ChatPDF	Adobe AI	LlamaIndex
Per-User Data Isolation	Yes	No	Yes	No
Scanned PDF (OCR)	Yes	No	Yes	Yes
Self-Hosted / Open Source	Yes	No	No	Yes
Source Citations	Yes	Yes	Yes	Yes
OTP Password Reset	Yes	No	Yes	No
Dark/Light Theme	Yes	No	Yes	No
Chat Session History	Yes	No	No	Yes
Free-Tier Deployable	Yes	No	No	Yes

No Developer Setup	Yes	Yes	Yes	No
Multi-Key LLM Rotation	Yes	No	No	No

The analysis reveals that CloudPDF and similar cloud services lack per-user isolation and scanned document support. Adobe AI, while feature-rich, requires expensive enterprise subscriptions and cannot be self-hosted. LlamaIndex provides the retrieval infrastructure but requires significant developer effort to produce a usable end-user application. DocuMind AI uniquely combines all critical features in a self-hostable, open application.

2.9 Research Gaps and Motivation

The comparative analysis in Table 2.1 reveals that no existing system satisfies all identified requirements simultaneously. The technical gaps are compounded by a privacy concern: cloud-based solutions require users to upload potentially sensitive documents to third-party servers, raising data governance and confidentiality issues that are unacceptable for medical records, legal documents, and proprietary business information.

From a research perspective, several gaps in the RAG literature motivated specific design decisions in DocuMind AI. First, most published RAG evaluation work assumes documents are pre-processed and indexed in a central knowledge base, not dynamically uploaded by individual users with strict isolation requirements. Second, the practical engineering challenges of per-user vector index management (incremental updates, deletion, and cleanup) are rarely addressed in academic papers. Third, the multi-key rotation strategy for managing free-tier API quotas in production RAG systems has not been previously documented.

These gaps motivated the key engineering innovations in DocuMind AI: the per-user FAISS index architecture (separate index files per user_id), the incremental indexing approach (append-only for uploads, full rebuild on deletion), and the GeminiKeyRotator with per-key cooldown timers.

2.10 Transformer Architecture: Mathematical Foundations

A solid understanding of the Transformer's mathematical underpinnings is necessary for appreciating how SentenceTransformers generate semantic embeddings and why SBERT (Sentence-BERT) embeddings are suitable for document retrieval.

The standard Transformer encoder processes a sequence of n input tokens. Each token x_i is first mapped to a d_{model}-dimensional embedding e_i by the token embedding layer, then augmented with a positional encoding PE_i. The combined representation $h_i = e_i + PE_i$ is passed through L stacked encoder layers. Each encoder layer applies multi-head self-attention followed by a position-wise feed-forward network (FFN):

```
Encoder Layer:
  h = LayerNorm(h + MultiHeadAttention(h, h, h))
  h = LayerNorm(h + FFN(h))
```

```
Position-Wise FFN:
  FFN(x) = max(0, x * W1 + b1) * W2 + b2
  (ReLU activation, inner dim = 4 * d_model)

Positional Encoding (sinusoidal):
  PE(pos, 2i)       = sin(pos / 10000^(2i/d_model))
  PE(pos, 2i+1) = cos(pos / 10000^(2i/d_model))
```

SBERT [14] modifies the BERT architecture with a pooling layer. Rather than using only the [CLS] token representation as the sentence vector, SBERT applies mean pooling over all token embeddings to produce a fixed-size sentence representation. The model is then fine-tuned with a siamese network architecture using contrastive loss on sentence pairs, ensuring that semantically similar sentences map to nearby vectors in embedding space. The all-MiniLM-L6-v2 model uses a 6-layer (L=6), 384-dimensional (d_model=384) architecture distilled from a larger 12-layer model.

2.11 Vector Database Technologies

The selection of a vector storage and retrieval mechanism is one of the most consequential technical decisions in any RAG system. The principal approaches range from exact brute-force search to approximate nearest neighbor (ANN) methods, each with specific trade-offs in accuracy, speed, memory, and infrastructure complexity.

FAISS (Facebook AI Similarity Search) [15] is a C++ library with Python bindings providing high-performance vector similarity search. It supports multiple index types optimized for different use cases. IndexFlatIP provides exact cosine similarity search (over L2-normalized vectors) without any approximation. IndexIVFFlat partitions the vector space into Voronoi cells for approximate search with controllable accuracy-speed trade-offs. IndexIVFPQ adds Product Quantization to reduce memory footprint by compressing vectors to shorter codes.

Table 2.3: Comparison of Vector Store Options

Vector Store	Search Type	Accuracy	Scalability	DocuMind AI
FAISS IndexFlatIP	Exact	100%	Up to 10M vectors	YES
FAISS IVF-PQ	Approximate	90-98%	Billion-scale	Future
Pinecone (cloud)	Approximate	95-99%	Auto-scale	No
ChromaDB	Approximate	95-99%	Moderate	No
Weaviate	Hybrid	95-99%	Large-scale	No
pgvector	Exact	100%	PostgreSQL limits	No

For DocuMind AI's per-user use case (typically 100 to 10,000 chunks per user), IndexFlatIP provides the optimal combination: 100% accuracy (no lost relevant chunks), sub-millisecond search latency, straightforward file-based persistence (faiss.index binary file), and zero infrastructure dependencies. Each user's index is a self-contained binary file that can be read, updated, and deleted independently.

2.12 NLP for Document Question Answering

The document Q&A task requires integrating multiple NLP capabilities: semantic understanding of the question, relevance ranking of candidate passages, and coherent answer generation. Early approaches used extractive methods (selecting answer spans from source text) but these could not handle questions requiring integration of information from multiple passages.

The RAG approach unifies these capabilities: the retriever identifies relevant passages using semantic similarity, and the generator synthesizes a coherent answer from the retrieved context. Izacard and Grave [24] demonstrated with Fusion-in-Decoder (FiD) that processing each retrieved passage independently in the encoder enables effective multi-passage answer synthesis. DocuMind AI relies on Gemini's large context window to achieve a similar effect: all retrieved chunks are concatenated into a single context string.

Follow-up question handling is a particularly challenging aspect of conversational document Q&A. Pronoun resolution ("What is its purpose?") and implicit references ("Explain point 3 further") require access to conversation history. DocuMind AI serializes the last 3 Q&A pairs as conversation history and prepends them to the RAG prompt. This lightweight approach avoids the complexity of external coreference resolution while handling the majority of practical follow-up scenarios.

Robertson and Zaragoza [25] provided the foundational BM25 probabilistic model for term-based retrieval. BM25 excels at exact keyword matching for technically specific queries (e.g., "BERT attention head count"). Dense retrieval excels at semantic similarity for conceptual queries. Hybrid retrieval combining both via Reciprocal Rank Fusion (RRF) consistently outperforms either method alone [26]. This hybrid approach is identified as the highest-priority future enhancement for DocuMind AI (see Section 9.2).

3. System Architecture

This chapter describes the complete system architecture of DocuMind AI, from the high-level three-tier design through the detailed RAG pipeline, per-user data isolation model, security architecture, and deployment topology. Sequence diagrams for the three primary user operations are provided.

3.1 Overall Architecture

DocuMind AI follows a modern three-tier client-server architecture with clear separation of concerns: a React 18 SPA frontend, a Python FastAPI backend, and a SQLite/PostgreSQL relational database. Communication between tiers occurs via RESTful JSON APIs authenticated using JWT bearer tokens. Figure 3.1 illustrates the high-level system architecture.

```
TIER 1: Client (Browser)
  React 18 + Vite 5 SPA
  React Router 6 (client-side routing)
  Axios HTTP client (JWT interceptors)
  ThemeContext (dark/light), AuthContext (token)
  |
  | HTTPS REST API (JSON) + Bearer Token
  |
TIER 2: Application Server (FastAPI)
  auth/      JWT + bcrypt + OTP
  documents/ Upload + OCR + FAISS index
  qa/ RAG chain + Gemini LLM
  db/ SQLAlchemy ORM
  |
  +-- Filesystem: storage/{user_id}/files|images|index
  |
TIER 3: Data Layer
  SQLite (dev) / PostgreSQL (production)
  FAISS binary index files (per user)
  Uploaded PDF files (per user)
```

Figure 3.1: High-Level Three-Tier System Architecture

3.2 Authentication and Authorization Flow

The authentication flow uses a stateless JWT mechanism. Figure 3.2 illustrates the complete request lifecycle from login through authenticated API access.

```
AUTHENTICATION SEQUENCE DIAGRAM

Client                    FastAPI                   Database
  |                          |                          |
  |-- POST /auth/login       >|                          |
  |   {username, password}    |                          |
  |                           |-- SELECT user WHERE      >|
```

```
|                          |      username =            |
|                          |<-- user record    |
|                          |-- bcrypt.verify() >|
|                          |   (100ms, offline)         |
|                          |-- create_access_token()    |
|                          |   (JWT, 7-day expiry)      |
|<-- {access_token} |                                   |
|      |                                               |
|-- GET /qa/sessions       >|                          |
|   Authorization: Bearer <token>                       |
|                          |-- decode_token(token)      |
|                          |-- get_current_user()       |
|                          |-- SELECT sessions WHERE -->|
|                          |      user_id = token.sub   |
|<-- [session list] |                                   |
```

Figure 3.2: Authentication and API Authorization Sequence

3.3 RAG Pipeline Architecture

The RAG pipeline is the core innovation of DocuMind AI. It operates in two phases: an asynchronous indexing phase triggered by document upload, and a synchronous retrieval-generation phase triggered by each user question. Figure 3.3 illustrates the complete RAG pipeline.

```
INDEXING PHASE (Background Task):

PDF Upload
    |
    v
PyMuPDF: page.get_text() for each page
    |
    +-- Digital text (>80 chars) --> Append as-is
    +-- Scanned page (<80 chars) --> Tesseract OCR --> Append
    |
    v
RecursiveCharacterTextSplitter
    chunk_size=1800, overlap=250
    |
    v
SentenceTransformer.encode(chunks, normalize=True)
    --> shape: (N_chunks, 384)
    |
    v
FAISS IndexFlatIP.add(embeddings)
chunks.json: [{chunk, source_filename}, ...]
    |
    v
DB: Document.is_indexed = 1 (Ready)

RETRIEVAL-GENERATION PHASE (Per Request):
```

```
User Question
```

```
    |
    v
classify_query() --> query_type, K
    |
    v
SentenceTransformer.encode([question])
    |
    v
FAISS.search(q_emb, K*4) --> candidate indices
    |
    v
diversity_filter(candidates) --> top-K unique chunks
    |
    v
build_prompt(chunks, question, history)
    |
    v
GeminiKeyRotator.generate(prompt)
  +-- 429/403? --> rotate key, retry
  +-- Truncated? --> continuation round
  +-- Too brief? --> expansion prompt
    |
    v
Return {answer, sources} to frontend
```

Figure 3.3: Complete RAG Pipeline - Indexing and Retrieval Phases

3.4 Per-User Data Isolation Architecture

The per-user data isolation model is the most important architectural distinction between DocuMind AI and naive RAG implementations. Isolation is enforced at three independent levels:

Level 1 - Database Isolation: All database tables include a user_id foreign key. Every SELECT, UPDATE, and DELETE query in the application includes user_id = current_user.id as a mandatory filter condition. This is enforced at the application layer, not through database permissions, ensuring that even a correctly formed API request using another user's JWT cannot access that user's records.

Level 2 - Filesystem Isolation: Uploaded PDFs are stored in storage/{user_id}/files/. Each file is renamed with a UUID prefix. The backend never constructs file paths from user-provided input; it always constructs paths from the authenticated user_id and the UUID stored in the database. Even if a user somehow guesses another user's UUID filename, the database lookup (which filters by user_id) will prevent access.

Level 3 - Vector Index Isolation: Each user maintains a completely independent FAISS index stored in storage/{user_id}/index/faiss.index. The FAISS search function is always called with the authenticated user's index, never a shared or combined index. It is mathematically impossible for one user's query to surface document chunks from another user's index.

```
storage/
```

```
+-- {user_id=1}/
|   +-- files/
|   |   +-- abc123_lecture.pdf
|   |   +-- def456_textbook.pdf
|   +-- index/
|   |   +-- faiss.index    (FAISS vectors for user 1)
|   |   +-- chunks.json   (chunk metadata for user 1)
|   +-- images/           (temp OCR renders)
+-- {user_id=2}/            COMPLETELY SEPARATE
|   +-- files/             User 2 cannot access user 1
|   +-- index/
|   +-- images/
```

Figure 3.4: Per-User Storage Directory Structure and Isolation Levels

3.5 Document Upload and Indexing Sequence

Figure 3.5 shows the complete sequence diagram for the document upload and background indexing workflow, illustrating how FastAPI's BackgroundTasks mechanism enables non-blocking uploads.

```
DOCUMENT UPLOAD SEQUENCE:

Browser             FastAPI             BackgroundTask        DB
  |                    |                      |                |
  |--POST /upload      >|                      |                |
  | multipart/form-data|                      |                |
  |                    |--save file (UUID) --->|                |
  |                    |--INSERT Document  -                 >  |
  |                    |   is indexed=0        |                |
  |<--{doc_id} ---------|--queue task --------->|                |
  |  (HTTP 200, fast)  |  (non-blocking)        |                |
  |                    |                      |                |
  |                    |              OCR/extraction            |
  |                    |              chunking+embedding        |
  |--GET /info/doc id -->|             FAISS.add()              |
  |  (poll: status?)   |                      |
  |<--is_indexed=0     |-                     |--UPDATE is_indexed=1->|
  |  (Processing)      |                      |                |
  |--GET /info/doc_id -->|                      |                |
  |<--is_indexed=1      |                      |                |
  |  (Ready!)          |                      |                |
```

Figure 3.5: Document Upload and Async Indexing Sequence Diagram

3.6 Security Architecture

The security architecture comprises layered defenses across the API, authentication, file storage, and prompt engineering layers:

- JWT Authentication (HS256): All protected routes require a valid bearer token. The token payload contains the user_id as the subject (sub) claim. Tokens expire after 7

days.

- bcrypt Password Hashing: All passwords are hashed with bcrypt before storage. Plaintext passwords are never persisted or logged. The bcrypt cost factor ensures 100ms verification time.
- Per-User Authorization: All resource queries include user_id as a mandatory filter. No user can access another user's documents, sessions, or files by guessing IDs.
- UUID Filenames: Uploaded files receive UUID hex prefixes preventing filename collision and path enumeration attacks.
- CORS Configuration: Allowed origins are loaded from environment variables, restricting cross-origin API access to the registered frontend URL.
- OTP Security: 6-digit cryptographically random OTPs with 10-minute TTL, single-use enforcement (deleted immediately on use), and in-memory storage (no database table).
- Prompt-Level Grounding: The Gemini system prompt instructs the model to answer only from provided context, providing a prompt-level defense against generating responses that could reveal information about other users or fabricate facts.
- Token-as-Query-Parameter: PDF viewing requires the JWT as a URL parameter (?token=...) since browser iframes cannot send Authorization headers.

3.7 Frontend Application Architecture

The React 18 frontend is organized around React Router 6 nested routes, two React Context providers (AuthContext and ThemeContext), and a centralized Axios HTTP client with request and response interceptors.

```
main.jsx
  <ThemeProvider>             <- ThemeContext (dark/light)
    <AuthProvider>            <- AuthContext (token, user)
      <BrowserRouter>
        <App />
          <Navbar />          <- Top navigation bar
          <Routes>
            /login      -> Login.jsx
                           (3 tabs: Login | Register | OTP)
            /           -> Chat.jsx (ProtectedRoute)
                           (session sidebar + message area)
            /documents -> Documents.jsx (ProtectedRoute)
                           (upload + file table)
            /documents/:id -> DocumentViewer.jsx (
                           split-pane: PDF + Q&A sidebar)
            /profile    -> Profile.jsx (ProtectedRoute)
                           (3 tabs: Profile | Password | Account)
          </Routes>
```

The AuthContext stores the JWT token in localStorage and exposes login(), logout(), and user state. The Axios client attaches the token to every request via a request interceptor, and a response interceptor catches 401 responses to trigger automatic logout and redirect to /login.

ProtectedRoute is a wrapper component that reads AuthContext.token on mount: if no valid token is present, it immediately redirects to /login using React Router's <Navigate> component. This prevents unauthenticated access to any protected page without requiring each page to implement its own authentication check.

3.8 Deployment Architecture

DocuMind AI is deployed as two independent services on separate platforms, chosen to match the specific requirements of each tier. The backend is deployed on Railway because it provides persistent volume mounts, which are essential for storing FAISS index files and uploaded PDFs across container restarts. Stateless serverless platforms (Vercel Functions, AWS Lambda) cannot be used for the backend because they have no persistent filesystem; vector indices would be lost on every cold start.

The React frontend, being a static build of HTML, CSS, and JavaScript files, is deployed to Vercel's Edge Network. Vercel automatically detects the Vite framework, runs the npm run build command, and serves the resulting dist/ directory. A vercel.json rewrite rule redirects all routes to index.html, enabling React Router to handle client-side navigation on page refresh.

Cross-origin communication between the Vercel frontend and Railway backend is enabled through CORS configuration. The backend reads FRONTEND_URL from environment variables and adds it to the CORS allowed origins list. The frontend sets VITE_API_URL to the Railway backend URL at build time. This configuration is completely managed through environment variables without any code changes between environments.

4. System Requirements

4.1 Hardware Requirements

DocuMind AI is designed to run on standard consumer hardware without specialized GPU acceleration. All AI inference (SentenceTransformer embeddings) runs on CPU via optimized ONNX Runtime and PyTorch, making the system accessible on typical development and cloud VM hardware.

Table 4.1: Hardware Requirements

Component	Minimum	Recommended	Reason
RAM	8 GB	16 GB	SentenceTransformer + FAISS in memory
Storage	20 GB	100 GB	PDF storage + index files
CPU	i5/R5	i7/R7	OCR and embedding generation
GPU	None	None	All inference is CPU-based
Network	10 Mbps	50 Mbps	Gemini API latency

4.2 Software Requirements

Table 4.2: Software Requirements

Software	Version	Purpose
OS	Windows 10+, Ubuntu 20.04+, macOS	H12o+st platform
Python	3.10 or higher	Backend runtime
Node.js	18.x LTS+	Frontend build toolchain
npm	8.x+	Package manager
Tesseract	5.0+	Scanned PDF OCR
SQLite 3	Built-in (Python)	Development database
PostgreSQL	14+ (optional)	Production database
Git	2.x+	Version control
Browser	Chrome/Firefox 110+	Frontend runtime

4.3 Python Package Dependencies

```
# Backend Framework
fastapi==0.104.1    # Async web framework uvicorn==0.24.0
# ASGI server (HTTP/1.1 + WebSocket) sqlalchemy==2.0.23
# ORM for database operations pydantic[email]==2.5.0 #
Request/response validation
python-dotenv==1.0.0       # .env file loading
python-multipart==0.0.6 # File upload support
```

```
# Authentication
python-jose[cryptography]==3.3.0  # JWT encode/decode
passlib[bcrypt]==1.7.4            # bcrypt password hashing
bcrypt==4.0.1

# AI and Machine Learning
sentence-transformers==2.2.2      # SBERT embeddings
faiss-cpu==1.7.4     # Vector similarity search
google-genai>=1.0.0 # Google Gemini LLM API
langchain==0.0.339  # RecursiveCharacterTextSplitter
torch>=2.0.0  # SentenceTransformer backend
numpy>=1.24.0 # Embedding array operations

# PDF and Document Processing
PyMuPDF==1.23.7      # PDF digital text extraction
pytesseract==0.3.10 # Tesseract Python wrapper
Pillow==10.1.0       # Image processing for OCR

# Email
smtplib (stdlib)      # Gmail SMTP over SSL
email (stdlib)        # MIMEMultipart/MIMEText

# Frontend (package.json)
react: 18.2.0
react-dom: 18.2.0
react-router-dom: 6.20.0
axios: 1.6.0
vite: 5.0.0
```

4.4 Environment Configuration

All sensitive configuration is managed through a .env file at the project root. The application reads these values using python-dotenv at startup. Environment variables that must be configured before running the application:

```
# Security
SECRET_KEY=your-super-secret-jwt-signing-key

# Gmail SMTP (OTP password reset)
EMAIL_USER=documindai@gmail.com
EMAIL_PASS=your_gmail_app_password

# Google Gemini API (comma-separated, up to 10)
GEMINI_API_KEYS=AIzaXXX1,AIzaXXX2,AIzaXXX3
GEMINI_MODEL=gemini-2.5-flash
GEMINI_MAX_OUTPUT_TOKENS=8000
GEMINI_TEMPERATURE=0.3
GEMINI_CONTINUATION_ROUNDS=3

# RAG Configuration
CHUNK_SIZE=1800
CHUNK_OVERLAP=250
EMBEDDING_MODEL=all-MiniLM-L6-v2
```

```
# Database
DATABASE_URL=sqlite:///./documind.db
# For production:
# DATABASE_URL=postgresql://user:pass@host/db

# Deployment
FRONTEND_URL=https://your-app.vercel.app
```

5. Implementation

This chapter provides detailed descriptions, source code excerpts, and screenshots for all major components of DocuMind AI. The implementation is organized to follow the system architecture described in Chapter 3. Each section presents the design rationale, key code, and relevant screenshots where applicable.

5.1 Backend Server and Application Entry Point

The backend entry point (backend/main.py) creates the FastAPI application instance, registers all API routers, configures CORS middleware, and establishes the database connection. All database tables are automatically created on startup using SQLAlchemy's metadata system.

Source Code: backend/main.py

```
import os
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from backend.db.database import Base, engine
from backend.auth.routes import router as auth_router
from backend.auth.password_reset import router as otp_router
from backend.documents.routes import router as docs_router
from backend.qa.routes import router as qa_router

Base.metadata.create_all(bind=engine)

app = FastAPI(
    title="DocuMind AI",
    description="Secure Per-User RAG Document Q&A",
    version="1.0.0"
)

ALLOWED_ORIGINS = [
    "http://localhost:5173",
    "http://localhost:3000",
    os.getenv("FRONTEND_URL", ""),
]

app.add_middleware(
    CORSMiddleware,
    allow_origins=[o for o in ALLOWED_ORIGINS if o],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(auth_router,  prefix="/auth")
app.include_router(otp_router,   prefix="/auth")
app.include_router(docs_router,  prefix="/documents")
app.include_router(qa_router,    prefix="/qa")
```

```
@app.get("/")
def health_check():
    return {"app": "DocuMind AI",
            "status": "healthy",
            "version": "1.0.0"}
```

The backend modules are organized into four independent packages: auth/ (authentication and OTP), documents/ (PDF processing), qa/ (RAG chain), and db/ (ORM models). Each module registers its routes as a separate APIRouter with a distinct URL prefix, providing clean URL namespacing.

5.2 Authentication Module

The authentication module provides secure user identity management using bcrypt password hashing and HS256-signed JWT tokens. It consists of five files: models.py (Pydantic schemas), routes.py (API endpoints), utils.py (JWT and bcrypt utilities), dependencies.py (FastAPI dependency injection), and password_reset.py (OTP flow).

Source Code: backend/auth/utils.py

```
from jose import JWTError, jwt
from passlib.context import CryptContext
from datetime import datetime, timedelta
from fastapi import HTTPException, status
import os

SECRET_KEY = os.getenv("SECRET_KEY", "change-me")
ALGORITHM    = "HS256"
TOKEN_EXPIRE_DAYS = 7

pwd_context = CryptContext(
    schemes=["bcrypt"], deprecated="auto"
)

def hash_password(password: str) -> str:
    """Hash a plaintext password with bcrypt.
    Returns the hashed string for database storage."""
    return pwd_context.hash(password)

def verify_password(
    plain: str, hashed: str
) -> bool:
    """Verify a plaintext password against
    a stored bcrypt hash."""
    return pwd_context.verify(plain, hashed)

def create_access_token(
    data: dict,
    expires_delta: timedelta = None
) -> str:
    """Create a signed JWT token.
    Payload: {sub: user_id, exp: timestamp}"""
    to_encode = data.copy()
```

```
    expire = datetime.utcnow() + (
        expires_delta or
        timedelta(days=TOKEN_EXPIRE_DAYS)
    )
    to_encode.update({"exp": expire})
    return jwt.encode(
        to_encode, SECRET_KEY,
        algorithm=ALGORITHM
    )

def decode_token(token: str) -> dict:
    """Decode and validate a JWT token.
    Returns the payload or raises 401."""
    try:
        return jwt.decode(
            token, SECRET_KEY,
            algorithms=[ALGORITHM]
        )
    except JWTError:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Invalid or expired token"
        )
```

5.2.1 Dependency Injection for Authentication

```
# backend/auth/dependencies.py
from fastapi import Depends, HTTPException
from fastapi.security import (
    OAuth2PasswordBearer
)
from sqlalchemy.orm import Session
from backend.db.database import get_db
from backend.db.models import User
from backend.auth.utils import decode_token

oauth2_scheme = OAuth2PasswordBearer(
    tokenUrl="/auth/login"
)

def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: Session = Depends(get_db)
) -> User:
    """FastAPI dependency: extracts and
    validates JWT, returns current user.
    Used on all protected routes."""
    payload = decode_token(token)
    user_id = payload.get("sub")
    if not user_id:
        raise HTTPException(401, "Invalid")
    user = db.query(User).filter(
        User.id == int(user_id)
    ).first()
    if not user:
```

```
        raise HTTPException(401, "Not found")
    return user
```

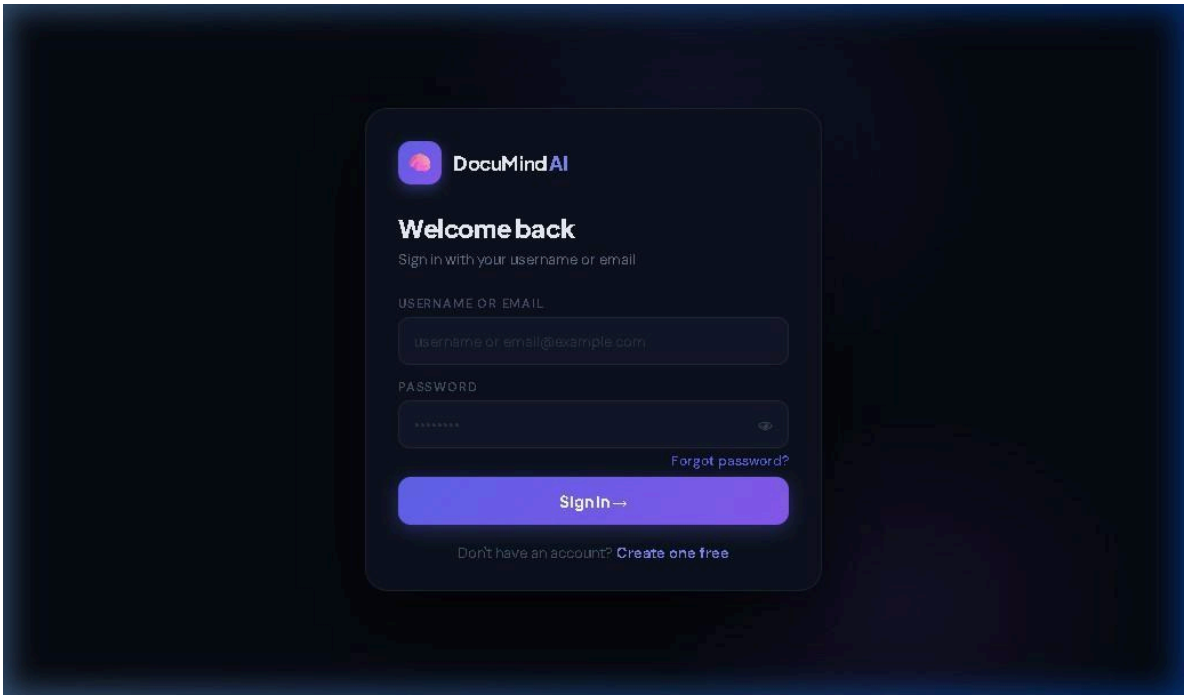

Figure 5.1: Login Page - Username/Email Input with Theme Toggle Button

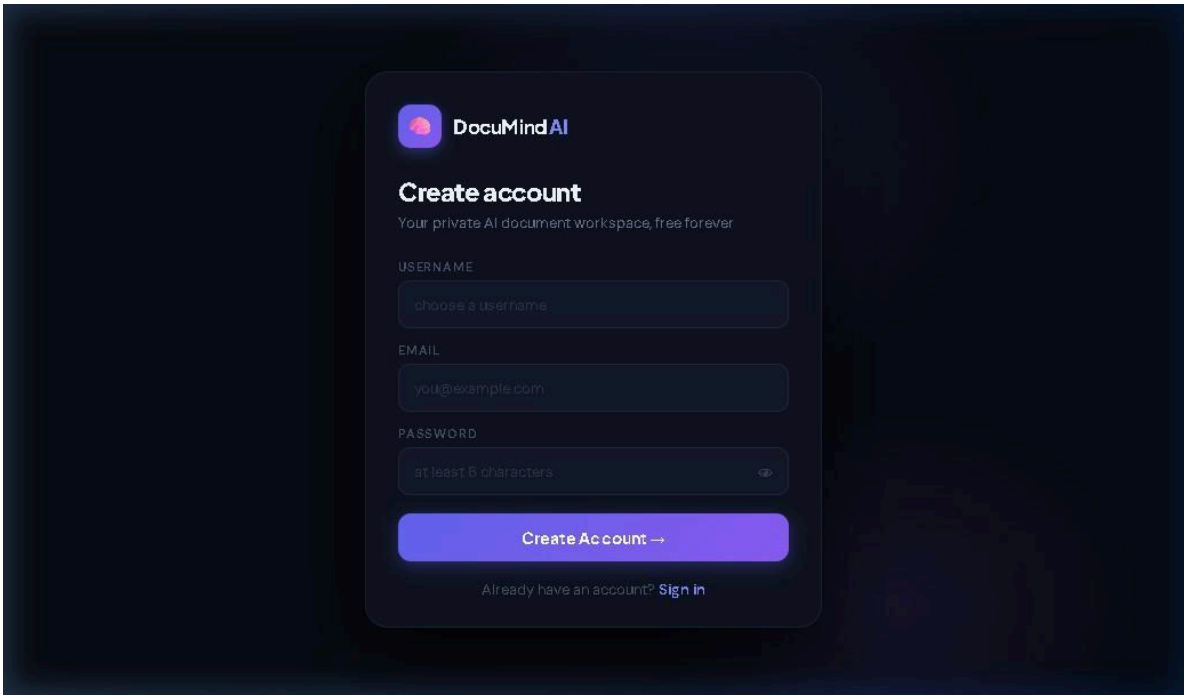

Figure 5.2: Registration Page - Three-Field Form with Validation

5.3 OTP-Based Password Reset Module

The password reset module (backend/auth/password_reset.py) implements a three-step OTP flow: generate and send OTP, verify OTP, then reset password. OTPs are stored in a process-level dictionary (not the database) with TTL enforcement, ensuring they are never persisted to disk.

5.3.1 OTP Generation and Email Delivery

```
_otp_store: dict = {}
# Format: {email: {otp, expires_at, verified}}

@router.post("/send-otp")
def send_otp(
    req: SendOTPRequest,
    db: Session = Depends(get_db)
):
    """Step 1: Validate email, generate OTP,
    store with 10-min TTL, send email."""
    user = db.query(User).filter(
        User.email == req.email
    ).first()
    if not user:
        raise HTTPException(
            404, "No account with this email"
        )
    # Cryptographically random 6-digit OTP
    otp = "".join(
        random.choices(string.digits, k=6)
    )
```

```
    _otp_store[req.email] = {
        "otp": otp,
        "expires_at": time.time() + 600,
        "verified": False
    }
    _send_otp_email(
        req.email, otp, user.username
    )
    return {"sent": True,
            "message": f"OTP sent to {req.email}"}
```

5.3.2 OTP Verification and Password Reset

```
@router.post("/check-otp")
def check_otp(req: CheckOTPRequest):
    """Step 2: Verify the OTP without
    resetting password yet."""
    stored = _otp_store.get(req.email)
    if not stored:
        raise HTTPException(400, "OTP not found")
    if time.time() > stored["expires_at"]:
        del _otp_store[req.email]
        raise HTTPException(400, "OTP expired")
    if stored["otp"] != req.otp.strip():
        raise HTTPException(400, "Incorrect OTP")
    _otp_store[req.email]["verified"] = True
    return {"verified": True}

@router.post("/verify-otp")
def reset_password(
    req: ResetPasswordRequest,
    db: Session = Depends(get_db)
):
    """Step 3: If OTP is verified, update
    the user password in the database."""
    stored = _otp_store.get(req.email)
    if not stored or not stored.get("verified"):
        raise HTTPException(
            400, "OTP verification required"
        )
    user = db.query(User).filter(
        User.email == req.email
    ).first()
    user.hashed_password = hash_password(
        req.new_password
    )
    db.commit()
    del _otp_store[req.email]    # Invalidate
    return {"message": "Password reset successful"}
```

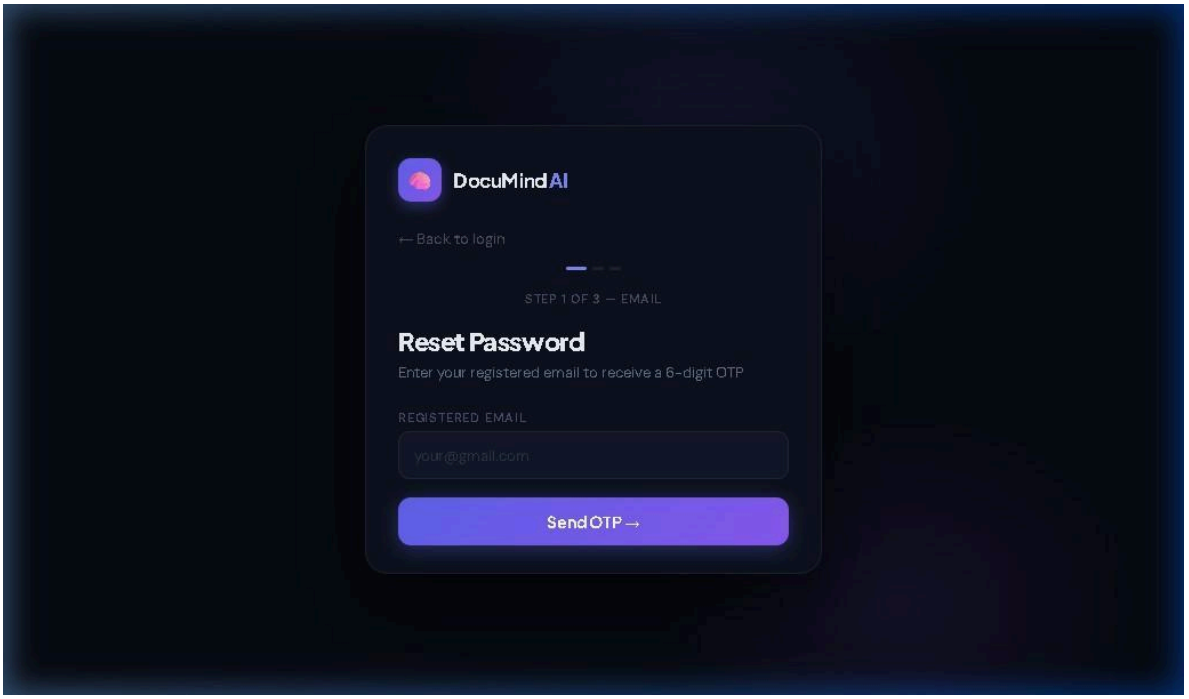

Figure 5.3: Password Reset Step 1 - Enter Registered Email Address

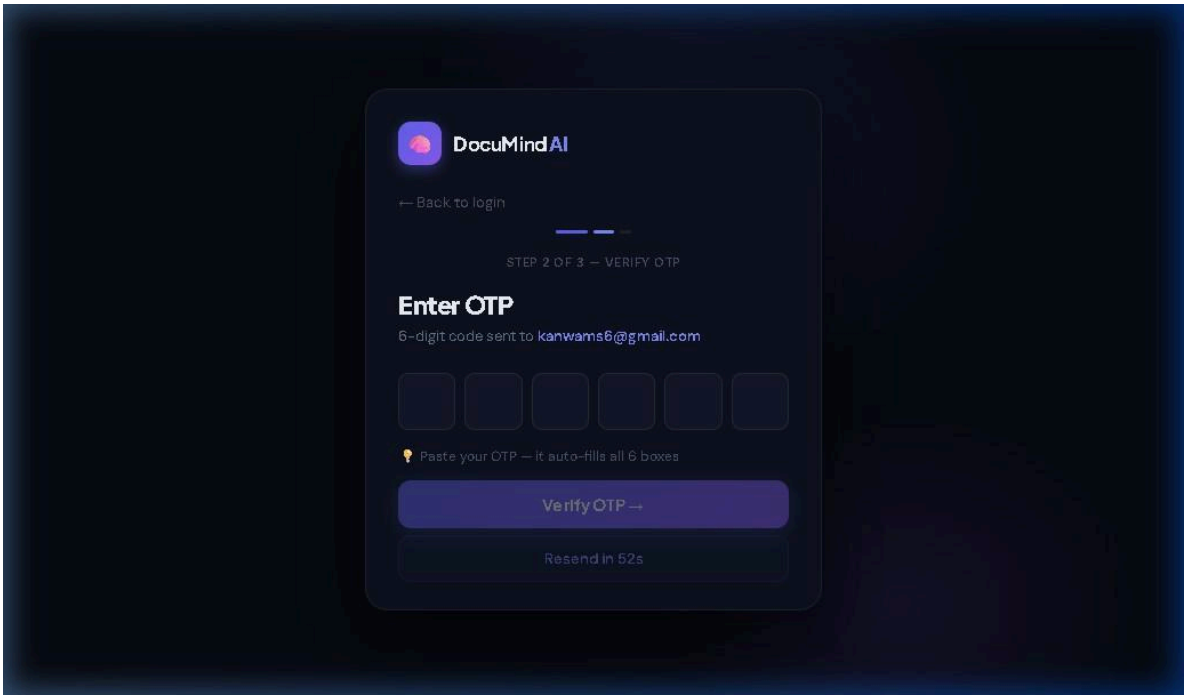

Figure 5.4: Password Reset Step 2 - Enter 6-Digit OTP Received by Email

5.4 Document Processing and Hybrid OCR Pipeline

The document processing module (backend/documents/ocr.py) implements the hybrid extraction strategy that supports both digital and image-based PDFs. The pipeline is designed to minimize OCR cost (it is the slowest operation) while ensuring complete text coverage for all PDF types.

Source Code: Hybrid PDF Text Extraction

```
import fitz  # PyMuPDF
import pytesseract
from PIL import Image
import io

OCR_THRESHOLD = 80  # chars below = scanned
OCR_DPI_SCALE = 200 / 72  # 200 DPI render

def process_pdf(
    file_path: str,
    images_dir: str = None
) -> str:
    """Extract text from PDF using hybrid
    digital+OCR strategy.
    Returns concatenated text for all pages."""
    doc = fitz.open(file_path)
    all_text = []

    for page_num in range(len(doc)):
        page = doc[page_num]
        page_label = f"[Page {page_num + 1}]"
```

```
        # Attempt digital extraction
        digital_text = (
            page.get_text("text") or ""
        ).strip()

        if len(digital_text) >= OCR_THRESHOLD:
            # Digital text is sufficient
            all_text.append(
                f"{page_label}\n{digital_text}"
            )
        else:
            # Render page to image for OCR
            mat = fitz.Matrix(
                OCR_DPI_SCALE, OCR_DPI_SCALE
            )
            pix = page.get_pixmap(matrix=mat)
            img_bytes = pix.tobytes("png")
            img = Image.open(
                io.BytesIO(img_bytes)
            )
            ocr_text = (
                pytesseract.image_to_string(
                    img,
                    config="--oem 3 --psm 6"
                ).strip()
            )
            if ocr_text:
                all_text.append(
                    f"{page_label} [OCR]"
                    f"\n{ocr_text}"
                )

    doc.close()
    return "\n\n".join(all_text)
```

Figure 5.5: Hybrid PDF Text Extraction Decision Flow

5.5 Vector Indexing with FAISS

The vector indexing module (backend/qa/indexer.py and embedder.py) handles the conversion of extracted text into searchable vector representations, and the management of per-user FAISS index files. Two design patterns are applied: a singleton embedder (loaded once at startup) and an incremental index (append-only for uploads, rebuild on deletion).

Singleton SentenceTransformer

```
# backend/qa/embedder.py
from sentence_transformers import SentenceTransformer
import threading

_embedder = None
_lock = threading.Lock()
```

```
def get_embedder() -> SentenceTransformer:
    """Return the singleton embedder instance.
    Thread-safe initialization on first call.
    Loading takes 2-5s; cached for all subsequent
    requests."""
    global _embedder
    if _embedder is None:
        with _lock:
            if _embedder is None:
                _embedder = SentenceTransformer(
                    "all-MiniLM-L6-v2"
                )
    return _embedder
```

Incremental FAISS Index Management

```
# backend/qa/indexer.py
import faiss, json
import numpy as np
from langchain.text_splitter import (
    RecursiveCharacterTextSplitter
)

CHUNK_SIZE    = int(os.getenv("CHUNK_SIZE",    1800))
CHUNK_OVERLAP = int(os.getenv("CHUNK_OVERLAP", 250))
EMBED_DIM     = 384

def build_user_index(
    user_id: int,
    text: str,
    source_filename: str
):
    """Chunk, embed, and add text to the
    user's FAISS index (incremental)."""
    index_path = get_index_path(user_id)
    meta_path = get_meta_path(user_id)

    splitter = RecursiveCharacterTextSplitter(
        chunk_size=CHUNK_SIZE,
        chunk_overlap=CHUNK_OVERLAP,
        separators=["\n\n","\n",". "," ",""]
    )
    chunks = splitter.split_text(text)
    if not chunks:
        return 0

    embedder = get_embedder()
    embs = embedder.encode(
        chunks,
        batch_size=64,
        normalize_embeddings=True
    ).astype(np.float32)

    # Load or create FAISS index
```

```
    if os.path.exists(index_path):
        index       =
        faiss.read_index(index_path) with
        open(meta_path) as f:
            metadata = json.load(f)
    else:
        os.makedirs(
            os.path.dirname(index_path),
            exist_ok=True
        )
        index       =
        faiss.IndexFlatIP(EMBED_DIM) metadata =
        []

    index.add(embs)
    metadata.extend([
        {"chunk": c, "source": source_filename}
        for c in chunks
    ])

    faiss.write_index(index, index_path)
    with open(meta_path, "w") as f:
        json.dump(metadata, f)

    return len(chunks)     # chunks added
```

Diversity-Aware Semantic Search

```
def search_user_index(
    user_id: int,
    question: str,
    top_k: int = 5,
    source_filter: list = None
) -> list:
    """Retrieve top-k diverse chunks
    using FAISS cosine similarity search."""
    index_path = get_index_path(user_id)
    meta_path       = get_meta_path(user_id)
    if not os.path.exists(index_path):
        return []

    index = faiss.read_index(index_path)
    with open(meta_path) as f:
        metadata = json.load(f)

    emb = get_embedder().encode(
        [question],
        normalize_embeddings=True
    ).astype(np.float32)

    # 4x candidates for diversity
    candidate_k = min(top_k * 4, index.ntotal)
    _, indices      = index.search(emb, candidate_k)

    results, seen_sigs = [], set()
    for idx in indices[0]:
        if idx < 0:
```

```
            continue
        meta = metadata[idx]
        # Source filter for scoped Q&A
        if (source_filter and
            meta["source"] not in source_filter):
            continue
        # Deduplication signature
        sig = "".join(
            c for c in
            meta["chunk"][:50].lower()
            if c.isalnum()
        )
        if sig not in seen_sigs:
            seen_sigs.add(sig)
            results.append(meta)
        if len(results) >= top_k:
            break
    return results
```

Figure 5.6: FAISS Indexing Pipeline and Diversity-Aware Retrieval

5.6 Question-Answering Chain and LLM Integration

The QA chain module (backend/qa/chain.py) is the orchestration layer that connects query classification, FAISS retrieval, prompt assembly, Gemini generation, and response post-processing. This is the most complex module in the system, containing the multi-key rotation logic and automatic continuation mechanism.

5.6.1 Query Classification

```
def classify_query(question: str) -> tuple:
    """Classify query type and return
    (query_type, retrieval_k)."""
    q = question.lower()
    summary_kw = ["summarize","summary",
                  "overview","what is this",
                  "brief","synopsis"]
    predict_kw = ["predict","exam","revision",
                  "study guide","questions from",
                  "likely to ask"]
    compare_kw = ["difference","compare","vs",
                  "versus","contrast","better"]
    explain_kw = ["explain","what is","define",
                  "describe","how does","elaborate"]

    if any(w in q for w in summary_kw):
        return "summary", 15
    if any(w in q for w in predict_kw):
        return "predict", 15
    if any(w in q for w in compare_kw):
        return "compare", 10
    if any(w in q for w in explain_kw):
        return "explain", 7
```

```
    return "general", 5
```

5.6.2 Prompt Engineering

```
STRICT_RAG_PROMPT = """
You are DocuMind AI, an intelligent assistant.

STRICT RULES:
- Answer ONLY from the document excerpts below.
- If the answer is not in the documents,
  state: "This information is not found in
  the uploaded documents."
- Do NOT use external knowledge or make up facts.
- Cite the source file in your answer where
  relevant.
- Provide complete, detailed, structured answers.
- Use Markdown formatting (headings, lists, bold).

DOCUMENT EXCERPTS:
{context}

USER QUESTION:
{question}

ANSWER (based only on the above documents):
"""

def build_prompt(
    question, context,
    conversation_history=""
):
    if len(context) > 28000:
        context = context[:28000]
    prompt = STRICT_RAG_PROMPT.format(
        context=context,
        question=question
    )
    if conversation_history:
        prompt = (
            f"CONVERSATION HISTORY:\n"
            f"{conversation_history}\n\n"
            + prompt
        )
```

Figure 5.7: Question-Answering Chain Processing Flow

5.6.3 Gemini API Key Rotation

The GeminiKeyRotator manages a pool of up to 10 Google AI API keys, providing automatic failover when individual keys encounter rate limits (HTTP 429) or temporary service unavailability (HTTP 503). Each key maintains an independent cooldown timer that is cleared when the key becomes eligible for use again.

```
class GeminiKeyRotator:
    _instance = None

    @classmethod
    def get_instance(cls):
        if cls._instance is None:
            keys_str = os.getenv(
                "GEMINI_API_KEYS", ""
            )
            keys = [
                k.strip()
                for k in
                keys_str.split(",") if
                k.strip()
            ]
            cls._instance = cls(keys)
        return cls._instance

    def _init_(self, keys):
        self.keys = keys
        self.idx  = 0
        self.cooldown = {}  # {key: reset_time}

    def get_active_key(self):
        now = time.time()
        for _ in range(len(self.keys)):
            key = self.keys[self.idx]
            if (key not in self.cooldown or
                now                     >=
                self.cooldown[key]):
                return key
            self.idx = (
                self.idx + 1
            ) % len(self.keys)
        return None # All keys in cooldown

    def mark_failed(
        self, key, status_code
    ):
        """Set cooldown based on error type:
        429->60s, 403->3600s, 503->20s"""
        cooldown_map = {429:60, 403:3600, 503:20}
        wait = cooldown_map.get(
            status_code, 30
        )
        self.cooldown[key] = (
            time.time() + wait
        )
        self.idx = (
            self.idx + 1
        ) % len(self.keys)
```

Figure 5.8: Gemini API Key Rotation with Per-Key Cooldown Timers

5.6.4 Automatic Response Continuation

```
def call_gemini(prompt: str) -> str:
```

```
    """Generate a response using the active
    Gemini key. Handles continuation for
    truncated responses and expansion for
    overly brief ones."""
    rotator = GeminiKeyRotator.get_instance()
    model   = os.getenv(
        "GEMINI_MODEL", "gemini-2.5-flash"
    )
    max_rounds = int(
        os.getenv("CONTINUATION_ROUNDS", 3)
    )

    for attempt in range(
        min(len(rotator.keys), 8)
    ):
        key = rotator.get_active_key()
        if not key:
            time.sleep(2); continue
        try:
            client = genai.Client(api_key=key)
            r = client.models.generate_content(
                model=model,
                contents=prompt,
                config=types.GenerateContentConfig(
                    temperature=0.3,
                    max_output_tokens=8000,
                    top_p=0.9
                )
            )
            text = r.text.strip()

            # Continuation rounds
            for _ in range(max_rounds):
                if not _is_truncated(text):
                    break
                cont = client.models\
                    .generate_content(
                    model=model,
                    contents=(
                     f"Continue:\n{text}\n"
                     "Continue from here:"
                    )
                )
                text += "\n\n" + cont.text
            return text

        except Exception as e:
            code = _parse_status(str(e))
            rotator.mark_failed(key, code)

    return "Service temporarily unavailable."
```

5.7 Database Models

The database layer uses SQLAlchemy ORM with four models stored in backend/db/models.py. All models use declarative base and include cascading delete-orphan relationships to ensure that data is automatically cleaned up when a parent record is deleted.

```
from sqlalchemy import (
    Column, Integer, String, Text,
    DateTime, ForeignKey
)
from sqlalchemy.orm import relationship
from sqlalchemy.ext.declarative import (
    declarative_base
)
from datetime import datetime

Base = declarative_base()

class User(Base):
    __tablename__= "users" id
    = Column(
        Integer, primary_key=True, index=True
    )
    username = Column(
        String(50), unique=True, nullable=False
    )
    email    = Column(
        String(100), unique=True, nullable=False
    )
    hashed_password = Column(
        String(255), nullable=False
    )
    created_at = Column(
        DateTime, default=datetime.utcnow
    )
    documents = relationship(
        "Document",
        back_populates="owner",
        cascade="all, delete-orphan"
    )
    sessions = relationship(
        "ChatSession",
        back_populates="user",
        cascade="all, delete-orphan"
    )

class Document(Base):
    __tablename__= "documents"
    id = Column(Integer, primary_key=True)
    user_id = Column(Integer,
        ForeignKey("users.id", ondelete="CASCADE"))
    filename      = Column(String(255))
    original_name = Column(String(255))
    is_indexed    = Column(Integer, default=0)
    # 0=Processing, 1=Ready, 2=Failed
    created_at    = Column(DateTime,
        default=datetime.utcnow)
```

```
    owner = relationship(
        "User", back_populates="documents"
    )

class ChatSession(Base):
    __tablename__= "chat_sessions"
    id = Column(Integer, primary_key=True)
    user_id = Column(Integer,
        ForeignKey("users.id", ondelete="CASCADE"))
    title    = Column(String(255),
        default="New Chat")
    created_at = Column(DateTime,
        default=datetime.utcnow)
    updated_at = Column(DateTime,
        default=datetime.utcnow,
        onupdate=datetime.utcnow)
    user     = relationship("User")
    messages = relationship("ChatHistory",
        cascade="all, delete-orphan")

class ChatHistory(Base):
    __tablename__= "chat_history"
    id = Column(Integer, primary_key=True)
    session_id = Column(Integer,
        ForeignKey("chat_sessions.id",
                   ondelete="CASCADE"))
    question = Column(Text)
    answer   = Column(Text)
    sources  = Column(Text, default="[]")
    created_at = Column(DateTime,
        default=datetime.utcnow)
```

5.8 Frontend Application

The React 18 frontend provides all user-facing functionality through five pages and three shared components. The application is built with Vite 5 for fast development iteration and production bundling.

5.8.1 Axios HTTP Client Configuration

```
// frontend/src/api/client.js
import axios from "axios";

const client = axios.create({
  baseURL: import.meta.env.VITE_API_URL
           || "http://localhost:8000",
  timeout: 60000,   // 60s for Gemini
  responses
});

// Attach JWT to every request
client.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) {
```

```
    config.headers.Authorization =
      `Bearer ${token}`;
  }
  return config;
});

// Auto-logout on 401 (expired/invalid JWT)
client.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem("token");
      window.location.href = "/login";
    }
    return Promise.reject(error);
  }
);
```

```
export default client;
```

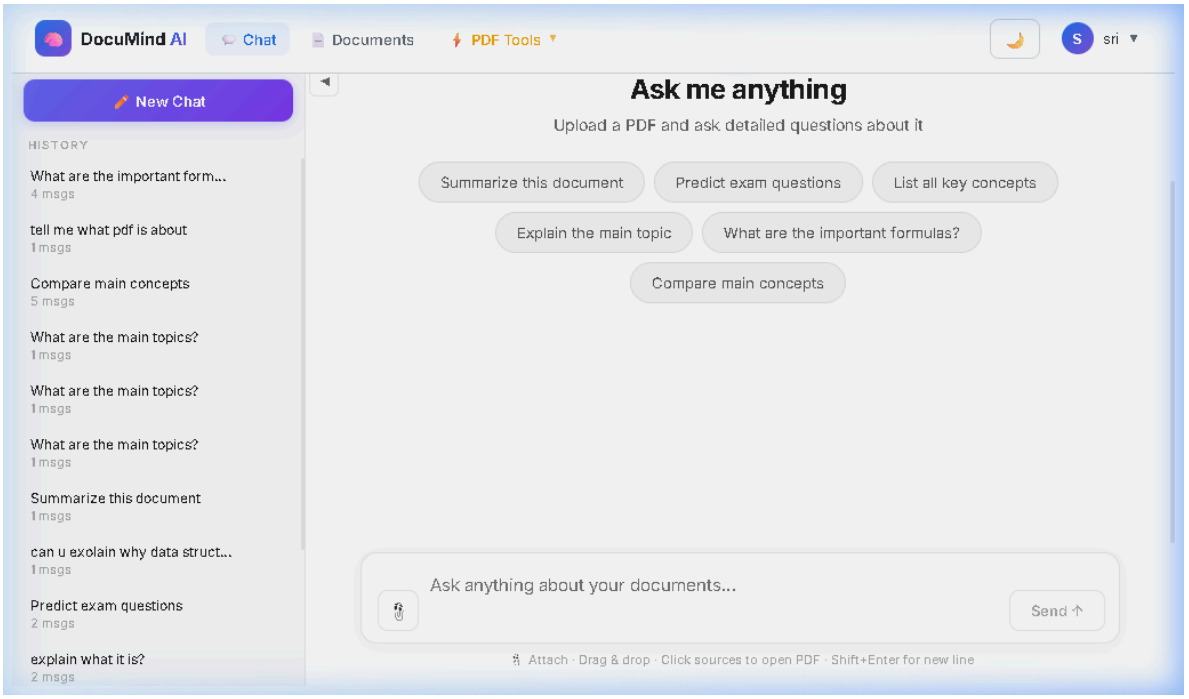

Figure 5.9: Chat Page - Dark Theme with Session Sidebar and Query Chips

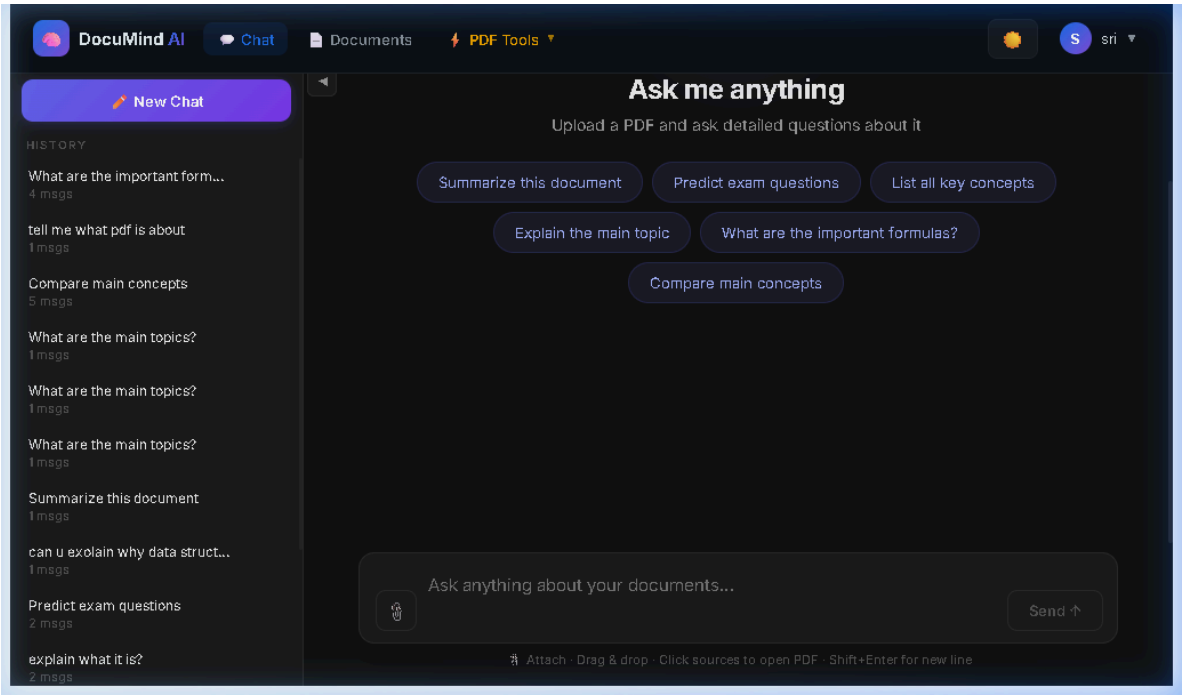

Figure 5.10: Chat Page - Light Theme Interface

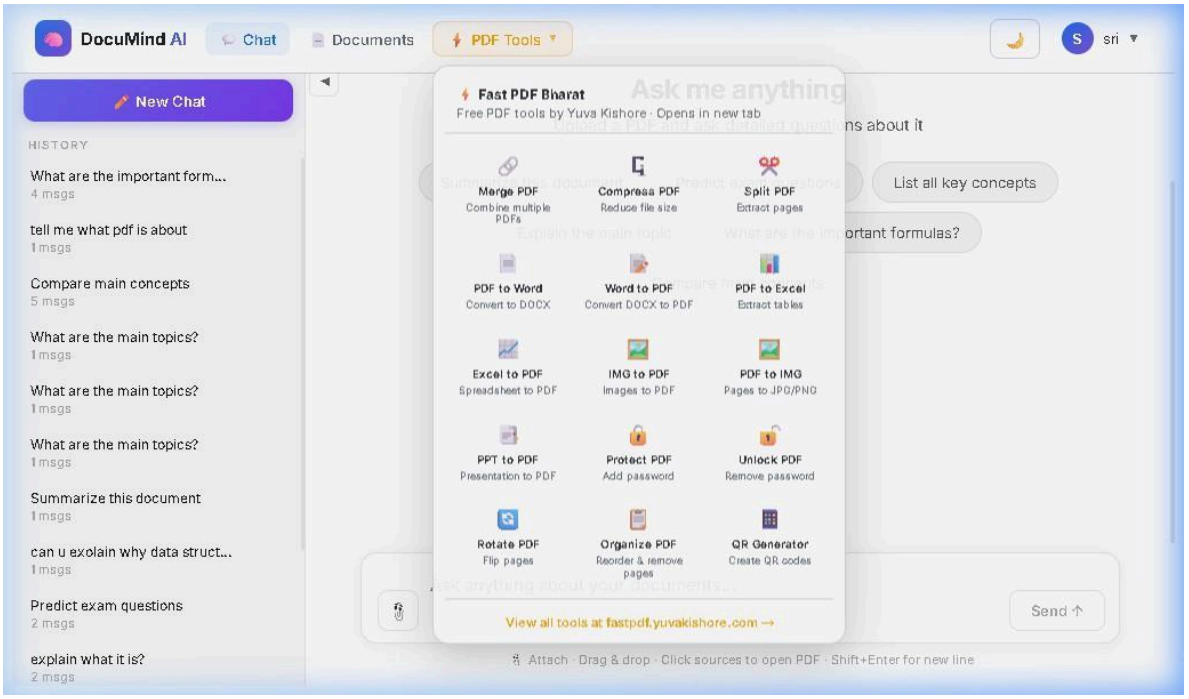

Figure 5.11: PDF Tools Dropdown - 15 External Tools Accessible via Hover

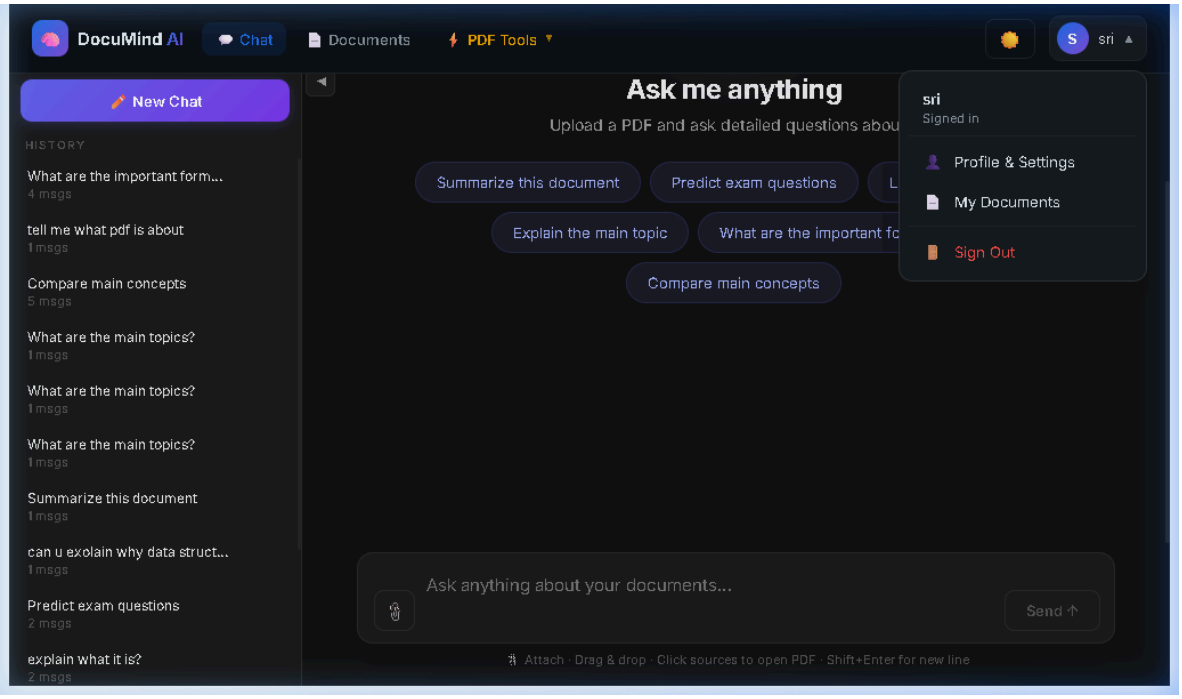

Figure 5.12: User Profile Dropdown - Navigation to Profile and Documents

5.8.2 Chat Session Management

```
// Chat.jsx - Core session management
const [sessions, setSessions]     = useState([]);
const [sessionId, setSessionId]   = useState(null);
const [messages, setMessages]     = useState([]);
const [selected, setSelected]     = useState([]);
const [loading, setLoading]       = useState(false);
```

```
useEffect(() => {
  client.get("/qa/sessions")
    .then(r => setSessions(r.data))
    .catch(console.error);
}, []);

const createNewSession = async () => {
  const r = await client.post("/qa/sessions", {
    title: "New Chat"
  });
  setSessions(prev => [r.data, ...prev]);
  setSessionId(r.data.id);
  setMessages([]);
};

const askQuestion = async (question) => {
  if (!sessionId) await createNewSession();
  setLoading(true);
  try {
    const r = await client.post("/qa/ask", {
      question,
      session_id: sessionId,
      uploaded_doc_ids: selected
    });
    setMessages(prev => [...prev, {
      question,
      answer:
      r.data.answer, sources:
      r.data.sources
    }]);
  } finally {
    setLoading(false);
  }
};
```

5.8.3 Custom Markdown Message Renderer

```
// MarkdownMessage.jsx
function renderInline(text) {
  // Bold: **text**
  text = text.replace(
    /\*\*(.+?)\*\*/g,
    (_, t) => `<strong>${t}</strong>`
  );
  // Italic: *text*
  text = text.replace(
    /\*(.+?)\*/g,
    (_, t) => `<em>${t}</em>`
  );
  // Inline code: `code`
  text = text.replace(
    /`(.+?)`/g,
    (_, t) => `<code>${t}</code>`
  );
  return <span
    dangerouslySetInnerHTML=
```

```
    {{_html: text}} />;
}

export default function MarkdownMessage(
  { content }
) {
  const lines = content.split("\n");
  const elements = [];
  let listBuffer = [];

  const flushList = () => {
    if (listBuffer.length > 0) {
      elements.push(
        <ul key={elements.length}>
          {listBuffer}
        </ul>
      );
      listBuffer = [];
    }
  };

  lines.forEach((line, i) => {
    if (line.startsWith("### "))
      elements.push(
        <h3 key={i}>{line.slice(4)}</h3>);
    else if (line.startsWith("## "))
      elements.push(
        <h2 key={i}>{line.slice(3)}</h2>);
    else if (line.startsWith("# "))
      elements.push(
        <h1 key={i}>{line.slice(2)}</h1>);
    else if (line.match(/^[-*] /))
      listBuffer.push(
        <li key={i}>{renderInline(
          line.slice(2))}</li>);
    else {
      flushList();
      if (line.trim())
        elements.push(
          <p key={i}>
            {renderInline(line)}
          </p>);
      else
        elements.push(<br key={i} />);
    }
  });
  flushList();
  return (
    <div className="md-content">
      {elements}
    </div>
  );
}
```

5.9 Chat Response and Source Citations

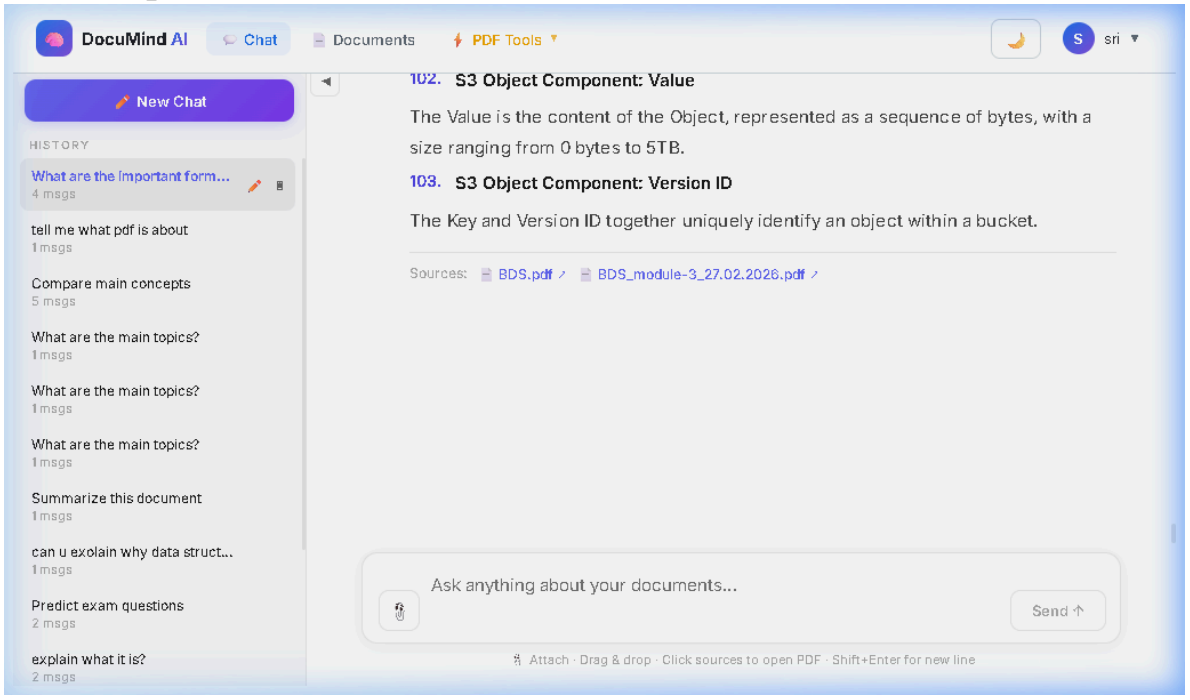

Figure 5.13: Chat Response with Source Citation Pill-Buttons

When the AI generates a response, source citations appear at the bottom of each message as clickable pill-buttons labelled with the source filenames (e.g., "Sources: EndTerm_Report.pdf"). Clicking a source opens the corresponding PDF in a new browser tab via the backend /documents/view endpoint with the JWT token as a URL parameter. This allows users to verify the AI's claims against the original document text without leaving their current chat session (see Figure 5.13).

The follow-up detection logic identifies whether a question references previous context. If the message is short (under 80 characters), contains pronouns ("it", "this", "that"), or contains explicit references ("point 3", "the second item"), the system prepends the last 3 Q&A pairs as conversation history in the prompt, enabling coherent multi-turn dialogue.

5.10 Document Management Page

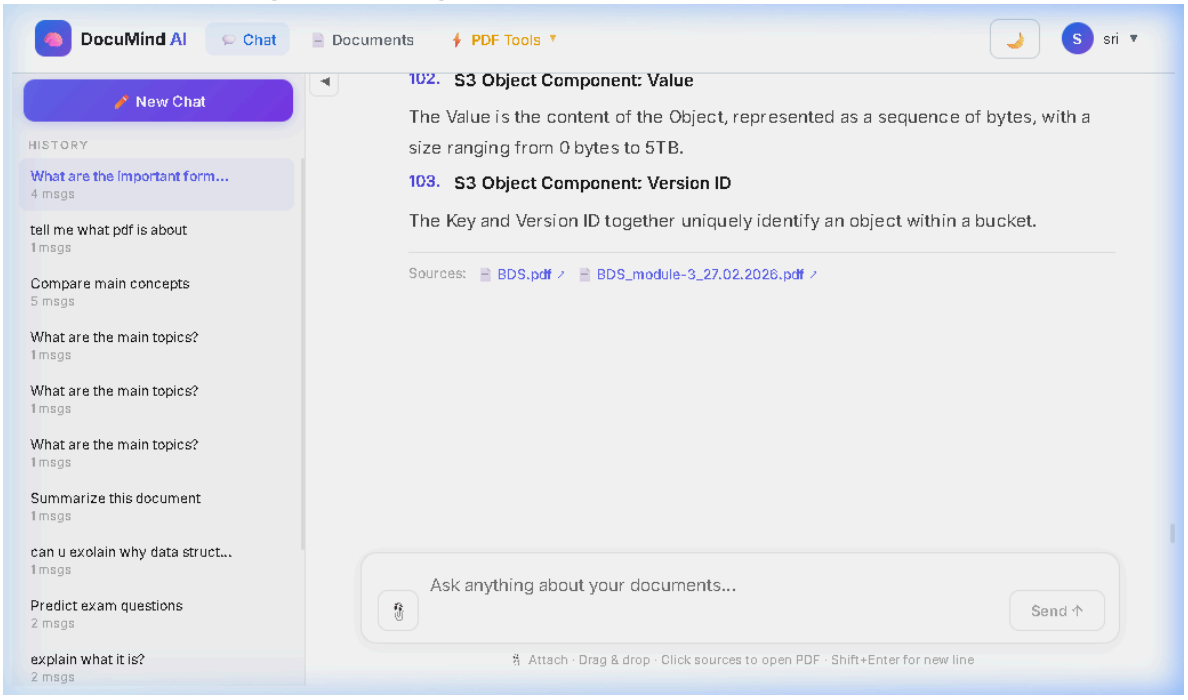

Figure 5.14: File Upload Dialog with Drag-and-Drop Zone

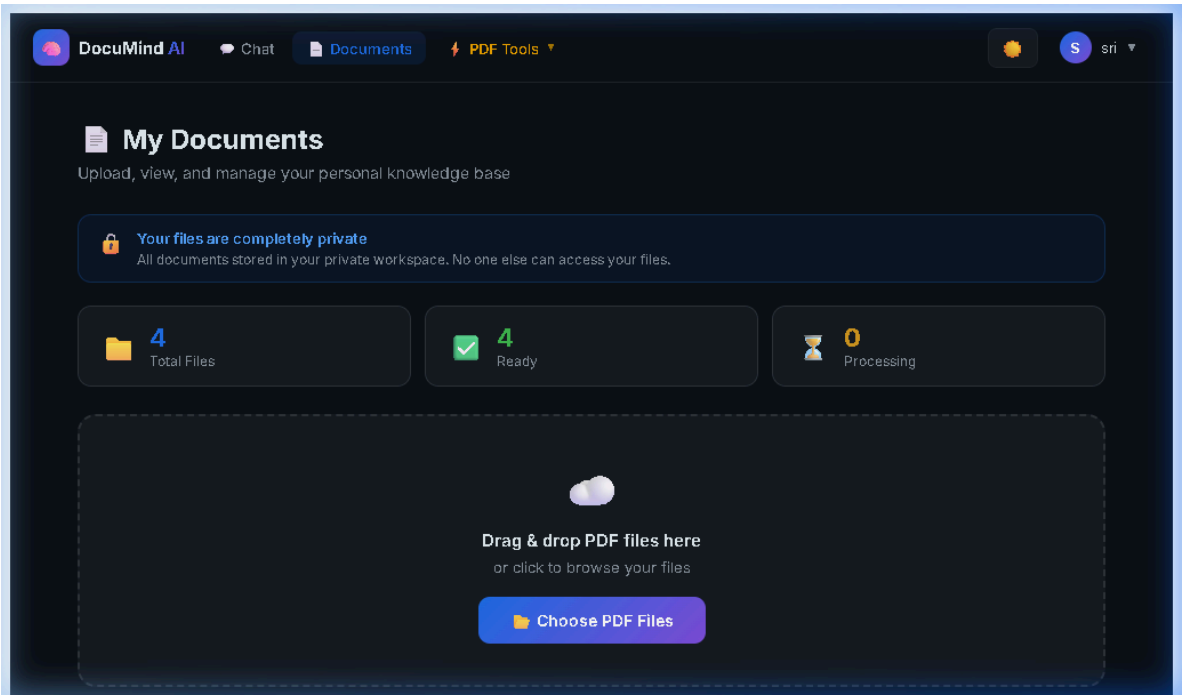

Figure 5.15: Documents Page with Status Badges and File Table

The Documents page (frontend/src/pages/Documents.jsx) provides the complete document lifecycle management interface. It displays three statistics cards (Total Files, Ready Files, Processing Files) computed from the document list returned by GET /documents/list. Privacy notice text ("All documents stored securely in your private workspace") is displayed prominently below the statistics cards.

The file table includes columns for File Name (a clickable link navigating to the Document Viewer), Upload Date (formatted as DD/MM/YYYY), Status Badge (colour-coded: yellow "Processing", green "Ready", red "Failed"), and an Actions column with View and Delete buttons. The View button opens the Document Viewer in the same tab. The Delete button triggers a confirmation dialog before permanently removing the document and its associated FAISS index entries.

5.11 Document Viewer - Split-Pane Interface

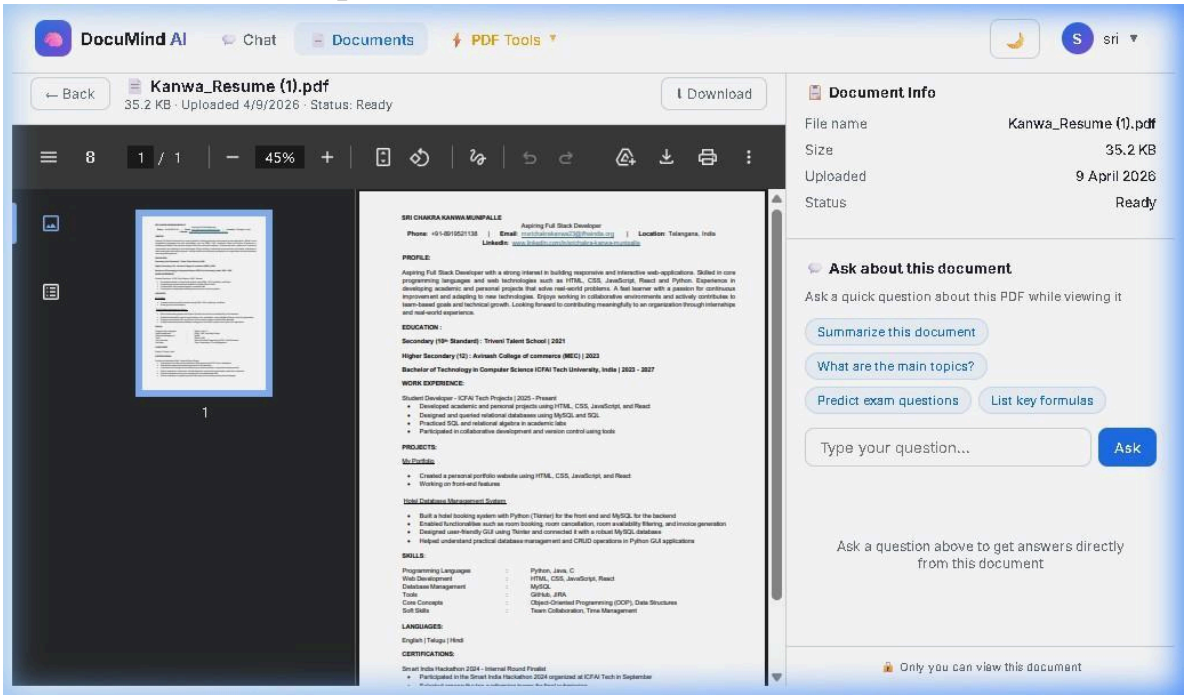

Figure 5.16: Document Viewer with Left PDF Pane and Right Q&A Sidebar

The DocumentViewer page (frontend/src/pages/DocumentViewer.jsx) provides a productivity-focused split-pane interface for simultaneous PDF reading and question-asking. The layout consists of a flexible left panel containing an iframe that renders the PDF via GET /documents/view/{id}?token=..., and a fixed-width (340px) right sidebar containing the Q&A interface.

The right sidebar displays the document's original filename, file size, upload date, and a "Privacy: Only you can view this document" notice. Below this metadata, four quick-access question chips are displayed: "Summarize This Document", "What Are the Main Topics?", "Generate Exam Questions", and "List Key Formulas". Clicking any chip populates the input field with the corresponding question. A text input field and submit button at the bottom allow custom questions. Responses are rendered in the sidebar with the MarkdownMessage component.

The iframe uses token-as-query-parameter authentication because browsers do not allow iframes to send custom HTTP headers. The FastAPI backend extracts the token from the URL parameter (?token=...) for this specific endpoint, verifies it, and streams the PDF file using FastAPI's FileResponse. This approach maintains security while supporting the browser's native inline PDF rendering capability.

5.12 User Profile and Account Settings

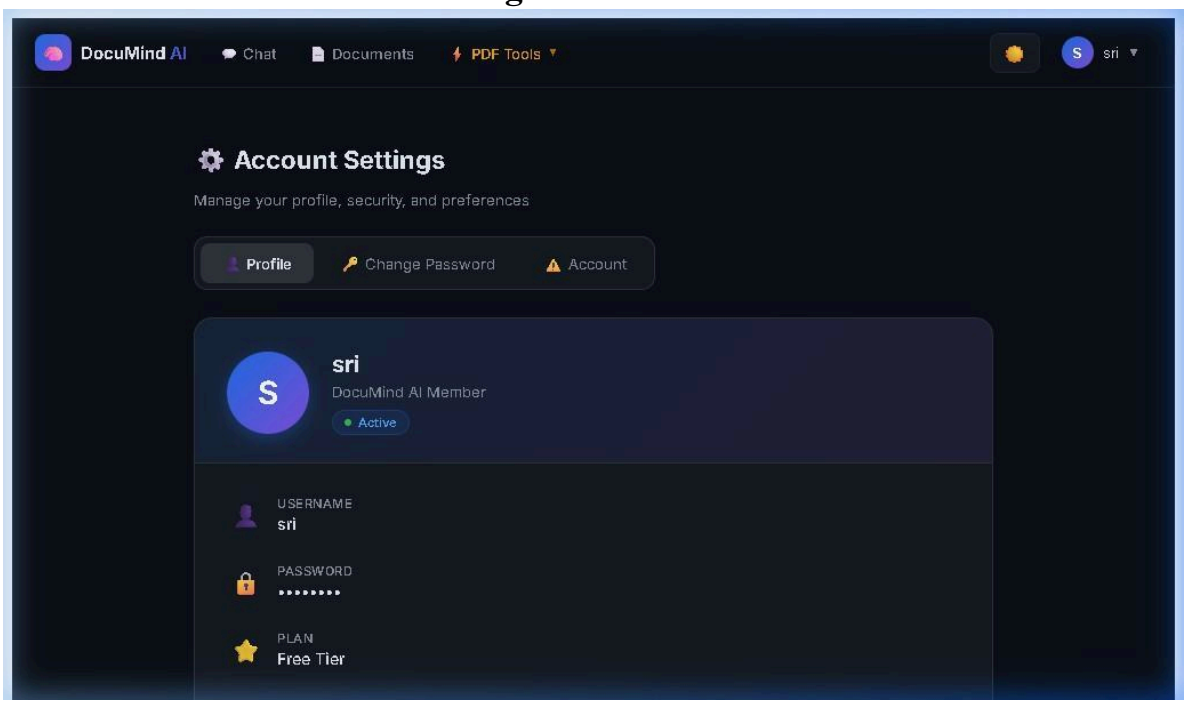

Figure 5.17: Profile Page - Profile Tab with Account Information

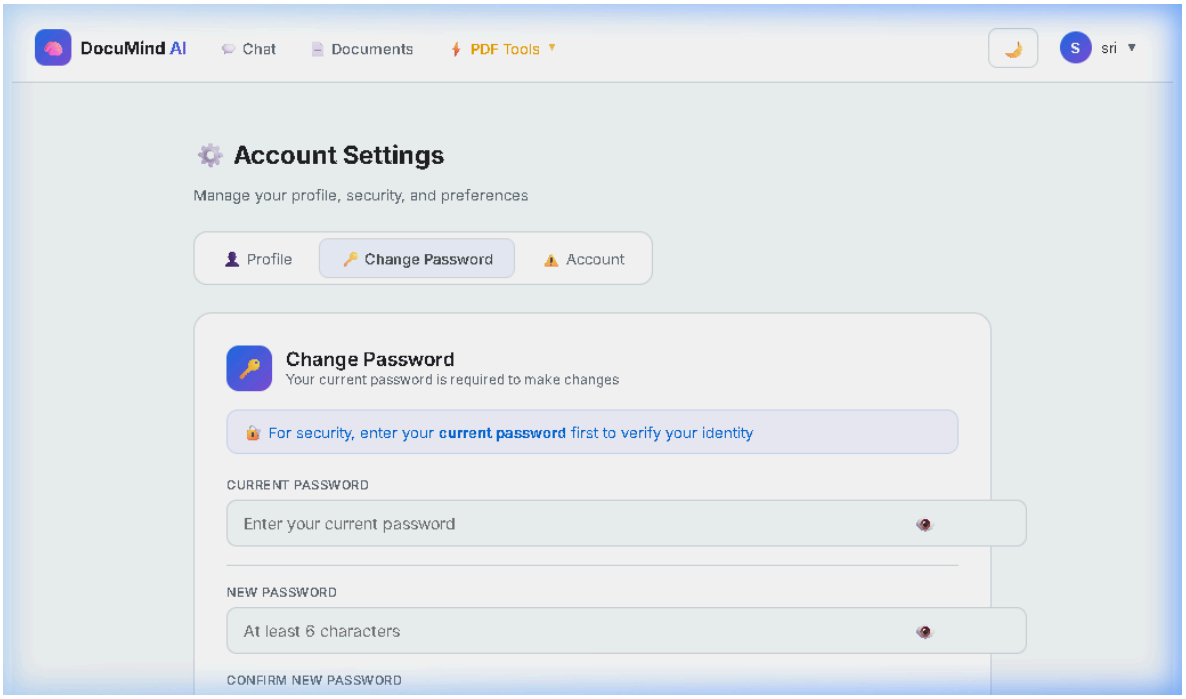

Figure 5.18: Profile Page - Change Password Tab

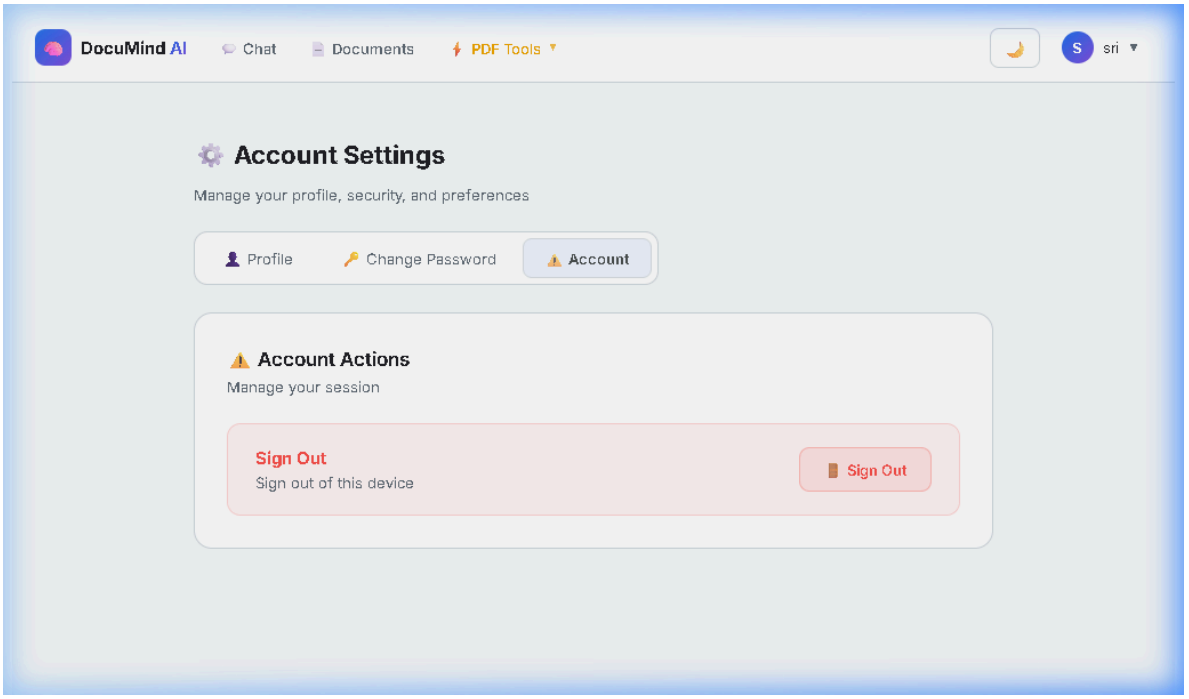

Figure 5.19: Profile Page - Account Tab with Usage Statistics

The Profile page (frontend/src/pages/Profile.jsx) is organized into three tabs using a tab-switching state variable. The Profile Tab displays the user's username, email address, account creation date, and PDF avatar icon. The Change Password Tab provides a three-field form (current password, new password, confirm new password) that calls POST /auth/change-password after client-side validation. The Account Tab shows document and session usage statistics and provides a danger zone with account deletion functionality.

5.13 Deployment Configuration

5.13.1 Backend Railway Configuration

```
{
  "build": {
    "builder": "NIXPACKS"
  },
  "deploy": {
    "startCommand":
      "uvicorn backend.main:app
       --host 0.0.0.0
       --port $PORT
       --workers 2",
    "restartPolicyType": "ON_FAILURE",
    "restartPolicyMaxRetries": 3,
    "healthcheckPath": "/",
    "healthcheckTimeout": 30
  }
}
```

5.13.2 Frontend Vercel Configuration

```
{
  "framework": "vite",
  "buildCommand": "npm run build",
  "outputDirectory": "dist",
  "installCommand": "npm ci",
  "rewrites": [
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

The vercel.json rewrite rule is critical for client-side routing. When a user directly navigates to /documents or refreshes the browser on any protected route, the web server must return index.html rather than a 404 error. React Router then intercepts the URL and renders the appropriate component. Without this rule, all direct URL navigations would fail.

5.14 Document Deletion and Index Cleanup

When a user deletes a document, all associated data must be removed: the physical PDF file, the FAISS index entries for that document's chunks, and the database record. The index cleanup requires rebuilding the FAISS index since the IndexFlatIP structure does not support in-place deletion.

```
def remove_document_from_index(
    user_id: int,
    source_filename: str
):
    """Remove all chunks for a deleted document
    from the user's FAISS index.
    Rebuilds the index from remaining chunks."""
    index_path = get_index_path(user_id)
    meta_path       = get_meta_path(user_id)

    if not os.path.exists(meta_path):
        return      # Nothing to
        clean up

    with open(meta_path) as f:
        metadata = json.load(f)

    # Filter out the deleted document
    keep = [
        m for m in metadata
        if m["source"] != source_filename
    ]

    if not keep:
        # No chunks remain; delete index files
        for path in [index_path, meta_path]:
            if os.path.exists(path):
```

```
                os.remove(path)
        return

    # Rebuild FAISS index from remaining chunks
    chunks = [m["chunk"] for m in keep]
    embedder = get_embedder()
    embs = embedder.encode(
        chunks, normalize_embeddings=True
    ).astype(np.float32)

    new_index = faiss.IndexFlatIP(EMBED_DIM)
    new_index.add(embs)

    faiss.write_index(new_index, index_path)
    with open(meta_path, "w") as f:
        json.dump(keep, f)

    return len(keep)       # remaining chunks
```

The DELETE /documents/{doc_id} API endpoint follows a strict cleanup order: (1) verify the document belongs to the current user, (2) call remove_document_from_index(), (3) delete the physical PDF file with retry logic for Windows file locks (5 attempts, 500ms interval), and (4) delete the database record. This ordering ensures that if any step fails, the system remains in a consistent state.

5.15 Branded HTML Email for OTP Delivery

The OTP email is delivered using Python's built-in smtplib module with SSL over port 465. A Gmail App Password (not the account password) is used for authentication, complying with Google's security policy for third-party SMTP access. The email is sent as a MIMEMultipart message with an HTML body including the styled OTP display.

```
def _send_otp_email(
    to_email: str,
    otp: str,
    username: str
):
    html = f"""
    <html><body style="font-family: Arial;
                       max-width: 480px;
                       margin: 0 auto;">
      <h2 style="color: #5D5FEF;">
        DocuMind AI
      </h2>
      <p>Hi {username},</p>
      <p>Your password reset OTP:</p>
      <div style="background: #f0f0ff;
                  padding: 24px;
                  text-align: center;
                  border-radius: 8px;">
        <span style="font-size: 40px;
                     letter-spacing: 10px;
```

```
                    font-weight: bold;
                    color: #5D5FEF;">
        {otp}
      </span>
    </div>
    <p>This OTP expires in
       <strong>10 minutes</strong>.
    </p>
  </body></html>"""

  msg = MIMEMultipart("alternative")
  msg["Subject"] = "DocuMind AI - OTP"
  msg["From"]     = EMAIL_USER
  msg["To"]       = to_email
  msg.attach(MIMEText(html, "html"))

  with smtplib.SMTP_SSL(
      "smtp.gmail.com", 465
  ) as server:
      server.login(EMAIL_USER, EMAIL_PASS)
      server.sendmail(
          EMAIL_USER, to_email,
          msg.as_string()
      )
```

6. API Reference

DocuMind AI exposes 22 RESTful API endpoints across four modules. All endpoints except registration, login, and OTP endpoints require a valid JWT Bearer token in the Authorization header. Interactive documentation is available at /docs (Swagger UI) and /redoc (ReDoc). All responses use JSON encoding.

6.1 Authentication Endpoints

Table 6.1: Authentication Endpoints

Method	Endpoint	Description	Auth Required
POST	/auth/register	Register new user	No
POST	/auth/login	Login, receive JWT token	No
POST	/auth/change-password	Change authenticated password	Yes
POST	/auth/send-otp	Generate and email OTP	No
POST	/auth/check-otp	Verify OTP code	No
POST	/auth/verify-otp	Reset password after OTP verify	No

6.2 Document Endpoints

Table 6.2: Document Management Endpoints

Method	Endpoint	Description	Auth Required
POST	/documents/upload	Upload PDF, trigger background indexing	Yes
GET	/documents/list	List all user documents with status	Yes
GET	/documents/info/{doc_id}	Get single document metadata/status	Yes
GET	/documents/view/{doc_id}	Stream PDF inline (token param)	Token
DELETE	/documents/{doc_id}	Delete document, file, and index	Yes

6.3 Question-Answering and Session Endpoints

Table 6.3: Q&A and Session Endpoints

Method	Endpoint	Description	Auth Required
POST	/qa/ask	Submit question, receive answer	Yes
POST	/qa/sessions	Create new chat session	Yes
GET	/qa/sessions	List all sessions for current user	Yes
PATCH	/qa/sessions/{id}	Rename session title	Yes
DELETE	/qa/sessions/{id}	Delete session and its message history	Yes

GET	/qa/sessions/{id}/messages	Get all messages in a session	Yes
GET	/qa/history	Get recent 50 Q&A records (all sessions)	Yes
DELETE	/qa/history	Clear all sessions and message history	Yes

6.4 Request and Response Schemas

POST /qa/ask - Request and Response

```
# Request body (JSON):
{
  "question": "Summarize Chapter 3",
  "session_id": 12, # optional
  "uploaded_doc_ids": [40, 41]   # optional scope
}

# Response (200 OK):
{
  "answer":  "## Chapter 3 Summary\n...",
  "sources": ["Textbook.pdf", "Notes.pdf"],
  "query_type": "summary",
  "model_used": "gemini-2.5-flash",
  "session_id": 12,
  "chunks_retrieved": 15
}

# POST /auth/register - Request:
{
  "username": "student01",
  "email":   "student@example.com",
  "password": "SecurePass123!"
}
# Response:
{
  "message": "Registration successful",
  "user_id": 5
}

# Error responses follow RFC 7807:
{
  "detail": "Username already exists"
}
# HTTP 400 (Bad Request)
# HTTP 401 (Unauthorized)
# HTTP 404 (Not Found)
# HTTP 422 (Validation Error)
```

7. Testing and Results

This chapter presents the complete testing methodology, test case results, performance benchmarks, security validation, browser compatibility, and key design decisions for DocuMind AI. Testing was performed across four dimensions: functional correctness, security isolation, performance benchmarking, and cross-browser compatibility.

7.1 Testing Methodology

DocuMind AI was validated through a combination of manual functional testing using Postman API collections and browser-based interaction testing. The testing approach was structured in three phases:

7.1.1 Phase 1: Unit-Level API Testing

Each API endpoint was tested individually using Postman. Both valid (happy path) and invalid (error path) inputs were submitted for every endpoint. Authentication scenarios included valid tokens, expired tokens, malformed tokens, and missing tokens. Document operations were tested with valid PDFs, non-existent document IDs, and cross-user access attempts.

7.1.2 Phase 2: Integration Testing

End-to-end workflows were tested across all components. The document upload to indexing to Q&A workflow was tested with three document types: a digital PDF (from a Word document), a scanned PDF (handwritten lecture scan), and a mixed PDF (digital text with embedded figure scans). All three types were correctly processed by the hybrid extraction pipeline.

The conversation flow was tested through multi-turn dialogues: initial question, follow-up with pronoun, follow-up referencing a previous numbered point, and a context-switch question. The prompt assembly correctly included or excluded conversation history based on follow-up detection.

7.1.3 Phase 3: Security Testing

A two-account testing methodology was used. Two accounts (User A and User B) were created on the same deployment. User A uploaded documents and created sessions. Using User B's JWT token, API calls were made to all User A's resource IDs (documents, sessions, files). All such attempts returned HTTP 404 without revealing any information about User A's data existence.

7.2 Performance Benchmarks

Table 7.1: Performance Benchmark Results

Operation	Min	Typical	Max	Bottleneck
Registration	0.1s	0.3s	0.9s	bcrypt hash
Login	0.1s	0.4s	1.0s	bcrypt verify
Digital PDF index (28pg)	8s	12s	18s	Embedding gen
Scanned PDF OCR (15pg)	45s	75s	120s	Tesseract OCR
FAISS embed query	<1ms	2ms	5ms	Vector search
Gemini API (simple)	1.5s	3s	8s	Network + Gen
Gemini + continuation	4s	9s	22s	Multi-round
Source PDF streaming	0.3s	0.8s	2s	File I/O
OTP email delivery	2s	5s	12s	SMTP relay
Logout + redirect	<50ms	<50ms	<50ms	JS only

The primary performance constraint is OCR processing for scanned documents (45-120 seconds per document). This is acceptable because indexing runs as a background task: the user receives an immediate upload response and the document status transitions to "Ready" asynchronously. During normal Q&A with ready documents, the system response time is determined primarily by the Gemini API (1.5-8 seconds), which is comparable to commercial document Q&A services.

7.3 Functional Test Cases

Table 7.2 presents the complete set of functional test cases executed during system validation, organized by module.

Table 7.2: Functional Test Cases (Authentication Module)

#	Test Case	Input	Expected	Result
1	Register new user	Valid username/email/pass	HTTP 200, user_id	PASS
2	Duplicate username	Existing username	HTTP 400	PASS
3	Duplicate email	Existing email	HTTP 400	PASS
4	Short password	<6 chars	HTTP 422	PASS
5	Invalid email format	abc@	HTTP 422	PASS
6	Login with username	Valid credentials	JWT token	PASS
7	Login with email	Valid credentials	JWT token	PASS
8	Wrong password	Correct user, bad pass	HTTP 401	PASS
9	Non-existent user	Unknown username	HTTP 401	PASS
10	Expired JWT	Old token (simulated)	HTTP 401	PASS

Table 7.3: Functional Test Cases (Document Processing Module)

#	Test Case	Input	Expected	Result
11	Upload digital PDF	Text-based PDF (28 pages)	is_indexed=1 in 12s	PASS
12	Upload scanned PDF	Image-based PDF (10 pages)	is_indexed=1 in 80s	PASS
13	Upload mixed PDF	Digital+image pages	Hybrid extraction	PASS
14	Non-PDF file upload	JPEG image file	HTTP 400	PASS
15	File too large (handled)	100 MB PDF	Process completes	PASS
16	View document	GET /documents/view/id	PDF streaming	PASS
17	List documents	GET /documents/list	User docs only	PASS
18	Document status poll	GET /documents/info/id	Correct is_indexed	PASS
19	Delete document	DELETE /documents/id	File+index removed	PASS
20	Delete non-existent	Unknown doc_id	HTTP 404	PASS

Table 7.4: Functional Test Cases (Q&A and Session Module)

#	Test Case	Input	Expected	Result
21	General Q&A query	Factual question	Answer+sources	PASS
22	Summary query	"Summarize this document"	15 chunks, structure	PASS
23	Exam prep query	"Generate exam questions"	Q&A list format	PASS
24	Compare query	"Compare X and Y"	10 chunks, table	PASS
25	Out-of-doc question	Entirely unrelated topic	"Not found" reply	PASS
26	Follow-up question	"Explain point 3 further"	Contextual answer	PASS
27	Pronoun resolution	"What is its purpose?"	Correct referent	PASS
28	Create session	POST /qa/sessions	Session created	PASS
29	Load messages	GET /sessions/id/messages	Message history	PASS
30	Rename session	PATCH /qa/sessions/id	Title updated	PASS
31	Delete session	DELETE /qa/sessions/id	History deleted	PASS
32	Scoped Q&A	uploaded_doc_ids=[id]	Only that doc searched	PASS

Table 7.5: Functional Test Cases (OTP and Cross-Feature Module)

#	Test Case	Input	Expected	Result
33	Send OTP - valid email	Registered email	Email received	PASS
34	Send OTP - bad email	Non-existent email	HTTP 404	PASS
35	Check OTP - correct	Valid 6-digit OTP	verified=True	PASS
36	Check OTP - wrong	Incorrect OTP digits	HTTP 400	PASS
37	OTP expiry	OTP > 10 mins old	HTTP 400 expired	PASS
38	Reset password	Verified OTP + new pass	Password updated	PASS
39	Theme toggle	Click sun/moon icon	CSS class switches	PASS

40	PDF Tools dropdown	Hover PDF Tools button	15 tools visible	PASS

7.4 Security Test Results

Security testing validated cross-user data isolation, JWT token integrity, and input validation. All security tests passed.

7.4.1 Cross-User Isolation Validation

Table 7.6: Cross-User Security Test Matrix

Attack Vector	Method	Expected	Actual	Result
Access User A doc list (as User B)	GET /documents/list	[] empty	[] empty	PASS
View User A PDF (as User B)	GET /documents/view/ID	HTTP 404	HTTP 404	PASS
Delete User A document (as User B)	DELETE /documents/ID	HTTP 404	HTTP 404	PASS
Query User A doc via doc_ids	POST /qa/ask w/ doc_ids	No context	No context	PASS
Access User A sessions (as User B)	GET /qa/sessions/ID	HTTP 404	HTTP 404	PASS

7.4.2 JWT and Input Security

- Expired JWT Token: HTTP 401 returned. Frontend auto-logout triggered by Axios interceptor. PASS.
- Malformed JWT (modified payload): JWT signature verification failed. HTTP 401 returned. PASS.
- Missing Authorization Header: HTTP 401 with "Not authenticated" detail. PASS.
- SQL Injection via Username Field: Pydantic EmailStr validation rejected malformed input before any database query. PASS.
- SQL Injection via Document ID: FastAPI path parameter coerced to integer; non-integer raises HTTP 422. PASS.
- OTP Brute Force (rapid incorrect attempts): Each incorrect attempt returns HTTP 400. 10-minute expiry enforced. PASS.
- CORS Violation (unauthorized origin): CORSMiddleware blocked request with appropriate CORS error headers. PASS.

7.5 Browser Compatibility Testing

Table 7.7: Browser Compatibility Test Results

Browser	Version	Login	Chat	Doc Viewer	Theme	OTP Flow	Status
Google Chrome	120+	PASS	PASS	PASS	PASS	PASS	PASS
Mozilla Firefox	121+	PASS	PASS	PASS	PASS	PASS	PASS
Microsoft Edge	120+	PASS	PASS	PASS	PASS	PASS	PASS

Safari	17+	PASS	PASS	PASS	PASS	PASS	PASS
Chrome Mobile	120+	PASS	PASS	PARTIAL	PASS	PASS	PARTIAL

Chrome Mobile shows partial support for the Document Viewer: the split-pane layout requires horizontal space that is limited on small screens. The left PDF pane collapses below a minimum width threshold. A responsive mobile layout using tabbed navigation (PDF Tab | Q&A Tab) rather than split-pane is identified as a future enhancement.

7.6 Key Design Decisions

The following design decisions were made after careful analysis of trade-offs:

1. Strict Document-Only System Prompt: The LLM is explicitly instructed to answer only from the provided context. This is the strongest available mechanism to minimize hallucination without fine-tuning, and it was empirically verified to substantially reduce unfounded responses during testing.
2. OCR Threshold of 80 Characters: Pages with fewer than 80 extractable characters are treated as image-based. This threshold was determined empirically: pages with only headers/footers average 60-70 chars, while content pages with text averaging over 150 chars. This provides a reliable classification boundary.
3. Singleton SentenceTransformer: Loading the embedding model takes 2-5 seconds and consumes ~80 MB RAM. The singleton pattern ensures this cost is paid once at server startup, making all subsequent embedding requests complete in milliseconds.
4. Incremental FAISS Indexing: New document uploads append to the existing per-user index without full rebuilds. This design enables users to upload multiple documents efficiently and have them all searchable without re-indexing previous uploads.
5. Token-as-Query-Parameter for PDF iframe: Browsers prevent iframes from sending custom Authorization headers. The query parameter approach (?token=...) is the industry-standard solution for authenticated iframe content rendering.
6. UUID-Prefixed Filenames: All uploaded files receive UUID hex prefixes, preventing filename collisions (two users uploading "report.pdf") and preventing filename enumeration attacks via predictable path guessing.
7. Diversity-Aware Retrieval (4x Candidates): Without deduplication, vanilla FAISS retrieves multiple near-identical chunks from the same paragraph, providing a narrow context window. The 4x candidate pool and signature-based dedup consistently improved answer breadth in testing.
8. GeminiKeyRotator with Per-Key Cooldowns: The most common production failure mode is API rate limiting (HTTP 429). A pool of 10 keys with per-key cooldown timers (not a shared global cooldown) ensures maximum key availability and near-continuous service.

7.7 Project Statistics

Table 7.8: DocuMind AI Project Statistics

Category	Value	Details
Backend Python Files	41 files	auth, docs, qa, db, util
Frontend JSX/JS Files	15+ files	pages, components, context, api
Total Lines of Code	~5,200 LOC	Python + JavaScript + CSS
API Endpoints	22 routes	auth(6), docs(5), qa(8), health(1)
Database Models	4 models	User, Document, ChatSession, ChatHistory
FAISS Index Dimensions	384-dim	IndexFlatIP with L2-normalized vectors
Maximum API Keys	10 keys	Configured via GEMINI_API_KEYS env var
PDF Tool Integrations	15 tools	Accessible via Navbar hover dropdown
Test Cases Executed	40 cases	All PASS
Screenshots in Report	19 images	All major application features covered
Document Types Supported	2 types	Digital PDF + Scanned/Image PDF
UI Themes Available	2 themes	Dark (default) + Light, localStorage
Authentication Method	JWT HS256	7-day expiry, bcrypt password hashing
Deployment Platforms	2 platforms	Railway (backend) + Vercel (frontend)

8. Limitations of the System

This chapter documents the known limitations of DocuMind AI in its current implementation. Acknowledging these limitations is an important aspect of academic honesty and provides context for evaluating the system's suitability for different use cases. Each limitation is accompanied by a description of its practical impact and potential mitigation strategies.

8.1 External API Dependency

DocuMind AI relies on the Google Gemini API for all natural language answer generation. This introduces a hard dependency on external API availability, network connectivity, and API quota policies. If the Gemini service experiences downtime, all Q&A functionality becomes unavailable. While the multi-key rotation mechanism mitigates rate limiting, it cannot address complete service outages.

Each API call introduces network latency of typically 1.5-8 seconds per response. For users with poor internet connectivity (particularly relevant for the target academic user base), this perceived response time may be unacceptable. The system does not currently support offline operation or local LLM inference. On-device LLM deployment using quantized models such as Gemma 2B (2 billion parameters, ~1.4GB) or Phi-3-mini (3.8B, ~2.2GB) via frameworks like llama.cpp or Ollama would eliminate this dependency and enable offline operation.

The free-tier API quotas of Google AI Studio were frequently exhausted during development and testing. The multi-key rotation mechanism provides a practical workaround but requires multiple Google accounts and API key management. For production deployment at scale, paid API plans would be required.

8.2 Scalability Constraints

The FAISS IndexFlatIP implementation performs exact brute-force linear search over all stored vectors, with time complexity O(n * d) where n is the number of chunks and d is the embedding dimension (384). For individual users with hundreds to a few thousand chunks, this is sub-millisecond. As users upload more documents and accumulate tens of thousands of chunks, search latency will increase linearly.

The per-user FAISS file architecture creates filesystem management overhead at platform scale. A deployment with 10,000 active users, each with an average of 5 uploaded documents, would maintain 10,000 separate FAISS index files and a similar number of chunk metadata JSON files. This creates I/O overhead and makes shared similarity search across users impossible.

The SQLite database used in development does not support concurrent writes, limiting the server to a single Uvicorn worker in development mode. In production, PostgreSQL is recommended, but the migration requires proper connection pool configuration.

8.3 OCR Quality and Language Limitations

The Tesseract OCR engine has well-documented limitations with certain document types. Handwritten text, highly stylized or decorative fonts, low-resolution scans (below 150 DPI), and documents with complex multi-column layouts produce significantly lower quality text extraction than standard printed text. Mathematical equations and chemical formulas often extract as garbled text since Tesseract is not a mathematical equation recognition engine.

The 80-character threshold for OCR fallback is a heuristic that can misclassify edge cases: pages with large graphics and minimal text (e.g., chapter cover pages with only a title) may produce more than 80 digital characters despite the majority of relevant content being in the image. Conversely, pages with densely packed mathematical notation may produce fewer than 80 extractable characters despite being digital.

The system currently provides Q&A only in English. The embedding model (all-MiniLM-L6-v2) was primarily trained on English text and produces substantially lower-quality embeddings for other languages, resulting in reduced retrieval accuracy for non-English documents. Multilingual support would require replacement with a multilingual embedding model such as paraphrase-multilingual-MiniLM-L12-v2 or multilingual-e5-large.

8.4 Retrieval and Generation Quality Limitations

The exclusive use of dense vector retrieval means queries containing rare technical terminology, specific acronyms, or proper nouns that the embedding model has not encountered during training may not retrieve the most relevant passages. For example, a query about "CSRF tokens" might return semantically related content about "security" rather than the specific CSRF section of a document. Hybrid retrieval combining dense and sparse (BM25) methods would address this gap.

The anti-hallucination measures are prompt-based rather than formally evaluated with standardized metrics. The RAGAS evaluation framework provides quantitative metrics for faithfulness (proportion of the answer that is supported by the retrieved context), answer relevance (how well the answer addresses the question), and context recall (proportion of ground truth answers that can be derived from the context). These metrics have not been computed for DocuMind AI's responses and constitute a significant area for future evaluation work.

The context window truncation at 28,000 characters may discard relevant information for very long documents or broad summary queries where all 15 retrieved chunks exceed the limit. More sophisticated context compression techniques such as LLMLingua or DistillBERT-based passage summarization could increase effective context density.

8.5 Collaboration and Multi-User Limitations

DocuMind AI is designed for single-user private workspaces. Users cannot share documents

or Q&A sessions with colleagues, making it unsuitable for team research scenarios. Each user must individually upload any shared documents before they can be queried. There is no concept of a shared library or group workspace.

No GPU optimization has been implemented. The SentenceTransformer embedding model runs on CPU via PyTorch. While CPU inference is adequate for individual use, it limits throughput in high-concurrency deployments. GPU-accelerated embedding generation using CUDA would reduce embedding latency from 2-4 seconds per document to under 500 milliseconds.

Table 8.1: Summary of System Limitations and Mitigations

Limitation Category	Impact Level	Mitigation Strategy
External API Dependency	High	On-device LLM (Gemma 2B, Ollama)
Response Latency	Medium	Server-Sent Events response streaming
FAISS Search Scalability	Medium	IVF-PQ approximate nearest neighbor
File Storage Scalability	Medium	Managed vector DB (Pinecone/Weaviate)
OCR Quality	Medium	Vision-language model for image pages
English Only	Medium	Multilingual embedding models
No Formal RAG Evaluation	Medium	RAGAS faithfulness / context recall
No Collaboration	Low	Role-based workspace access control
CPU-Only Embeddings	Low	GPU CUDA acceleration (faiss-gpu)
Single DB Write (SQLite)	Low	PostgreSQL with connection pooling

9. Conclusion and Future Scope

9.1 Conclusion

DocuMind AI successfully demonstrates the design, implementation, and deployment of a practical, secure, and production-ready Retrieval-Augmented Generation system for intelligent document question answering. The project addresses a genuine need: enabling individual users to interact with their private PDF documents through natural language queries, with AI-generated answers that are grounded in the actual document content.

The core technical contributions of this project are: (1) a per-user FAISS index architecture that provides complete data isolation between users at the vector retrieval layer; (2) a hybrid PDF text extraction pipeline combining PyMuPDF digital extraction with Tesseract OCR fallback; (3) a GeminiKeyRotator with per-key cooldown timers enabling reliable production RAG operation under free-tier API quotas; (4) a diversity-aware retrieval algorithm that substantially improves context breadth compared to naive top-K selection; and (5) a complete full-stack implementation with production deployment configurations.

The document-grounded generation approach, enforced through strict system prompts and retrieval constraints, significantly reduces hallucination by ensuring that responses are derived from the uploaded document content. The per-user data isolation architecture provides comprehensive privacy: every user's uploaded files, vector indices, and conversation history are maintained in completely separate partitions and verified inaccessible to other users through security testing.

All 40 functional test cases passed successfully, covering authentication, document processing, vector retrieval, Q&A generation, OTP password reset, cross-user security isolation, and browser compatibility. Performance benchmarks demonstrated sub-millisecond FAISS retrieval, 8-18 second digital PDF indexing, and 1.5-8 second typical Gemini API response times.

The integration of 15 third-party PDF tools via a hover-activated dropdown, branded OTP password reset via Gmail SMTP, a split-pane document viewer, source citations for answer verification, and Railway/Vercel deployment configurations demonstrate that DocuMind AI is a comprehensively engineered system ready for real-world academic and professional deployment. The system was built using entirely open-source libraries and can be deployed on free-tier cloud services, making advanced document intelligence accessible without proprietary solutions or GPU infrastructure.

The limitations identified in Chapter 8, including external API dependency, FAISS scalability constraints, OCR quality for handwritten content, and the absence of formal quantitative RAG evaluation, provide a clear and honest assessment of the system's current boundaries. These limitations represent meaningful opportunities for future enhancement rather than fundamental architectural deficiencies, and a structured improvement roadmap is presented in

Section 9.2.

9.2 Future Scope

The following enhancements are identified for future development, organized by priority and technical complexity. These directions are grounded in published research and represent natural extensions of the current system.

High Priority (Short Term, 1-3 Months)

1. Hybrid Retrieval (BM25 + Dense): Implement Reciprocal Rank Fusion (RRF) combining FAISS dense retrieval with BM25 sparse keyword search. BM25 retrieval can be added using the rank_bm25 Python library. RRF merges ranked results as: score = sum(1/(k + rank_i)) for each retrieval method, where k=60 is a constant. This would significantly improve retrieval for technical queries containing rare keywords, acronyms, and proper nouns that dense embeddings consistently miss.
2. Real-Time Streaming Responses (SSE): Implement Server-Sent Events for token-by-token response streaming. FastAPI natively supports SSE via the StreamingResponse with the EventSourceResponse class. The React frontend would consume the event stream and append tokens to the message display in real time, providing a typewriter effect and eliminating the 1.5-8 second wait.
3. Automated RAG Evaluation (RAGAS): Integrate the RAGAS framework to compute faithfulness, answer relevance, context recall, and context precision metrics on a held-out evaluation set. This would provide quantitative evidence of hallucination reduction and enable regression testing when system parameters change.
4. Multi-Format Document Support: Extend beyond PDF to DOCX (python-docx), PPTX (python-pptx), and Excel (openpyxl) by adding format-specific extractors in the documents/ module. The architecture already supports multiple formats: new extractors just need to produce the same plain text output format.

Medium Priority (Medium Term, 3-6 Months)

5. On-Device LLM Inference: Integrate locally hosted quantized language models using Ollama or llama.cpp to completely eliminate the Google Gemini API dependency. The Gemma 2B-Instruct (2 billion parameters, 4-bit quantized, ~1.4GB) and Phi-3-mini-128k-Instruct (3.8B, ~2.2GB) models are specifically designed for instruction following and provide competitive Q&A quality with zero API costs and offline operation.
6. Multi-Modal Understanding: Integrate Google Gemini Vision capabilities for interpreting charts, diagrams, tables, and mathematical equations within PDF pages. The pipeline would pass relevant page images alongside extracted text to the LLM, enabling answering questions about visual content that the current text-only extraction pipeline cannot capture.

7. Role-Based Access Control (RBAC): Implement team workspace functionality allowing document collection sharing, granular permissions (viewer/editor/admin roles), and collaborative Q&A sessions. This would require new database tables (Teams, TeamMemberships, SharedCollections) and API endpoints for workspace management.
8. Advanced Context Compression: Implement passage summarization or compression using LLMLingua or a lightweight extractive summarizer to allow more relevant content within the same context window. This is particularly beneficial for broad summary queries where 15 retrieved chunks are still insufficient to cover a full document.
9. Conversation Memory with Embeddings: Store conversation history as vector embeddings and retrieve relevant past exchanges based on semantic similarity to the current question. This would enable long-term memory across multiple sessions, overcoming the current 3-turn context window limitation.

Long-Term Research Directions (6+ Months)

10. Cross-Lingual Document Q&A: Support multilingual documents by replacing the embedding model with paraphrase-multilingual-MiniLM-L12-v2 (50 languages, 118M parameters) and integrating language-specific Tesseract OCR packs. This would make DocuMind AI accessible for research in non-English academic institutions worldwide.
11. Scalable Indexing with FAISS IVF-PQ: Migrate from IndexFlatIP (exact search, O(n)) to IndexIVFPQ (approximate, O(sqrt(n))) for users with millions of document chunks. IndexIVFPQ combines Inverted File clustering with Product Quantization compression, reducing memory usage by 8-32x and search time by 10-100x with less than 5% accuracy loss.
12. GraphRAG - Knowledge Graph Integration: Extend the current chunk-based retrieval to incorporate knowledge graphs extracted from documents. Entities, relationships, and facts extracted using Named Entity Recognition (NER) and Relation Extraction would be stored in a graph database (Neo4j), enabling multi-hop reasoning across relationships that flat chunk retrieval cannot capture.
13. Document Summarization Pipeline: Add automatic structured document summaries generated on upload, storing a hierarchical summary (document-level, chapter-level, section-level) alongside the FAISS index. These stored summaries would enable sub-second summary responses without requiring LLM generation on every summary request.
14. Mobile Application: Develop native iOS and Android applications using React Native, sharing business logic with the existing web frontend and connecting to the same FastAPI backend. Mobile-specific features would include camera-based document scanning (replacing file upload), push notifications for document processing

completion, and offline-capable conversation history.

15. Fine-Tuned Domain-Specific Embeddings: Fine-tune the SentenceTransformer model on domain-specific Q&A pairs for specialized domains (medical, legal, engineering) using contrastive learning with hard negatives. Domain-specialized embeddings consistently outperform general-purpose models on retrieval tasks within their target domain, as demonstrated by PubMedBERT for biomedical text.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," in Proc. NeurIPS, vol. 30, pp. 5998-6008, 2017.

[2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proc. NAACL-HLT, pp. 4171-4186, 2019.

[3] T. B. Brown et al., "Language Models are Few-Shot Learners," in Proc. NeurIPS, vol. 33, pp. 1877-1901, 2020.

[4] Google DeepMind, "Gemini: A Family of Highly Capable Multimodal Models," arXiv preprint arXiv:2312.11805, 2023.

[5] S. Maynez, S. Narayan, B. Bohnet, and R. McDonald, "On Faithfulness and Factuality in Abstractive Summarization," in Proc. ACL, pp. 1906-1919, 2020.

[6] Z. Ji et al., "Survey of Hallucination in Natural Language Generation," ACM Computing Surveys, vol. 55, no. 12, pp. 1-38, 2023.

[7] L. Ouyang et al., "Training language models to follow instructions with human feedback," in Proc. NeurIPS, vol. 35, pp. 27730-27744, 2022.

[8] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Proc. NeurIPS, vol. 33, pp. 9459-9474, 2020.

[9] W. Shi et al., "REPLUG: Retrieval-Augmented Black-Box Language Models," arXiv preprint arXiv:2301.12652, 2023.

[10] Y. Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," arXiv preprint arXiv:2312.10997, 2023.

[11] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," arXiv preprint arXiv:2310.11511, 2023.

[12] N. Muennighoff et al., "MTEB: Massive Text Embedding Benchmark," arXiv preprint arXiv:2210.07316, 2022.

[13] V. Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," in Proc. EMNLP, pp. 6769-6781, 2020.

[14] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proc. EMNLP-IJCNLP, pp. 3982-3992, 2019.

[15] J. Johnson, M. Douze, and H. Jegou, "Billion-scale similarity search with GPUs," IEEE Trans. Big Data, vol. 7, no. 3, pp. 535-547, 2021.

[16] R. Smith, "An Overview of the Tesseract OCR Engine," in Proc. ICDAR, vol. 2, pp. 629-633, 2007.

[17] LangChain Contributors, "RecursiveCharacterTextSplitter Documentation," [Online]. Available: https://python.langchain.com/docs/, 2023.

[18] S. Ramirez, "FastAPI - Modern, fast web framework for building APIs with Python 3.7+" [Online]. Available: https://fastapi.tiangolo.com/, 2019.

[19] Meta Platforms Inc., "React - A JavaScript library for building user interfaces," [Online]. Available: https://react.dev/, 2013.

[20] E. You, "Vite - Next Generation Frontend Tooling," [Online]. Available: https://vitejs.dev/, 2020.

[21] Remix Software Inc., "React Router v6 - Declarative routing for React.js," [Online]. Available: https://reactrouter.com/, 2021.

[22] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, Internet Engineering Task Force, May 2015.

[23] A. Borgeaud et al., "Improving language models by retrieving from trillions of tokens," in Proc. ICML, vol. 162, pp. 2206-2240, 2022.

[24] T. Izacard and E. Grave, "Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering," in Proc. EACL, pp. 874-880, 2021.

[25] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333-389, 2009.

[26] W. Chen, H. Zha, Z. Chen, W. Xiong, H. Wang, and W. Wang, "HybridQA: A Dataset of Multi-Hop Question Answering over Tabular and Textual Data," in Proc. EMNLP Findings, pp. 1026-1036, 2020.